

Measured, not estimated: what routing an agent to a cheaper model actually costs

A preregistered, paired, multi-turn measurement campaign

Amplifier Michael J. Jabbour

Microsoft, Office of the CTO

October 2, 2026

Abstract

An AI coding agent can hand some of its work to a cheaper model. Whether that saves money in a long conversation is hard to estimate, because providers cache the conversation separately for each model and a switch throws that cache away. We stopped estimating and measured. For each of 70 scripted 8–16-turn sessions, the plain host model, the shipped router (which re-chooses its model at the start of each turn), a “sticky” router that chooses a model once and never switches, and plain Sonnet (plus, on a 20 % subsample, a second plain copy) were launched within seconds of each other from one frozen workspace, each with its own cache key, twice on each of two host models: 1,036 sessions in all. Hypotheses, thresholds, split and decision rule were preregistered; the estimator was written after the data were collected. On the 23 held-out scenarios, with Fable 5.1 as host at our price table, sticky routing cost $0.56\times$ as much as the plain host (95 % confidence interval $0.49\text{--}0.64$), with turn-pass fraction inside the preregistered margin; plain Sonnet cost about the same ($0.58\times$). With Opus 5.5 as host, every routing arm cost more ($1.25\times\text{--}1.43\times$; $1.20\times\text{--}1.46\times$ without the quality filter); that result depends on Opus’s cheap cache reads and would reverse if they cost about 0.38 instead of 0.20 dollars per million tokens. The sticky policy cost less than the shipped one on both hosts. A savings model fitted on the training scenarios passed its preregistered pooled check on the held-out ones. The measurement cost \$3,527.29.

Contents

1	Summary	2
I	Fast judges: constructed and real	3
2	Fast judges on constructed cases	3
3	Fast judges on real decisions	8
II	What caching does to savings	9
4	Why estimates were not enough	9
5	What an earlier caching survey showed	10
6	How prompt caching works, and what a probe showed	11
III	Measuring savings	14
7	Design of the experiment	14
8	Running it safely	18
9	Confirmatory results [preregistered]	19
10	Exploratory results [screen]	24
11	More explorations of the campaign data [screen]	28
IV	Implications	33
12	What it means	33
13	Limitations and threats to validity	34
14	Reproducibility	35
A	Glossary	36
B	Additional tables	37

1 Summary

A coding agent such as Amplifier runs on a *host model*: the large model that answers each turn of a conversation. The fast-decisions bundle can *route* some of the work to a cheaper model (here Sonnet 5) when it judges the work easy enough. Routing looks like an obvious saving. It is not obvious once a conversation gets long, because of *prompt caching*: the provider keeps a cheap-to-reuse copy of the conversation *for each model separately*, so switching models means paying again to cache the whole history (Section 6).

This report answers one question with measurements rather than estimates: *what does routing an agent to a cheaper model actually cost, in money and in quality, over realistic multi-turn sessions?*

Key idea

What we found (test split, preregistered hypotheses; prices of Table 7; Section 9):

- **Table 5.1 host: Sonnet is cheaper.** Choosing the model once at the start of the session (*sticky*) cost $0.56\times$ the plain host (95 % CI 0.49–0.64), a saving of about 44 %, with turn-pass fraction inside the preregistered margin on the test scenarios. The shipped router, which re-chooses its model at the start of each turn, cost $0.63\times$; plain Sonnet for the whole session cost $0.58\times$.
- **Opus 5.5 host: routing to Sonnet cost more.** Sticky $1.25\times$, shipped $1.38\times$, plain Sonnet $1.43\times$ (pair reading); without the quality filter (arm reading) $1.20\times$, $1.32\times$ and $1.46\times$, that is 20–46 % more. This depends on Opus’s cache-read price: at about \$0.38 per million tokens instead of \$0.20, sticky would break even (Section 6).
- **Sticky cost less than shipped on both hosts:** $0.862\times$ on Fable and $0.917\times$ on Opus. The preregistered test was only that sticky costs no more than shipped; on Fable the margin is small (upper bound 0.994).
- **The savings model passes a pooled check.** Fitted on 47 training scenarios, it predicted the held-out total within its 90 % interval and its intervals covered the held-out pairs at the preregistered rate. Its per-host and per-task figures are not reliable (Section 10.6).

All 4 preregistered hypotheses were confirmed, under both readings of “non-inferior” quality that the preregistration allows (Section 9). Three cautions travel with the result.

- **Quality is close to the line.** Sticky on Fable passed the 5-point turn-pass margin on the test scenarios by 0.0035; across all 70 scenarios the margin was not shown (-0.021 , 95 % CI -0.058 to 0.012), and routed Fable arms passed the final hidden tests 3.6–4.3 points less often (exploratory; Section 10.3).
- **The router adds little beyond “use Sonnet”.** 248 of 280 sticky sessions ran Sonnet only, and plain Sonnet cost about the same on Fable (Section 12).
- **Scope.** Sessions are scripted, at most 16 turns long, on one provider and one price table (Section 13).

How to read this report. The report follows the whole study arc in four parts.

- **Part I, fast judges.** What a decision judge is, how 10 judges did on constructed cases (Section 2) and on real decisions rebuilt from agent traces (Section 3), and the four bundle defects those studies found and fixed.
- **Part II, what caching does to savings.** Why estimates were not enough (Section 4), what an earlier caching survey showed (Section 5), and how prompt caching works, with a direct probe of the provider (Section 6).

- **Part III, measuring savings.** How the design evolved and how big it had to be (Section 7), how it was run safely (Section 8), the confirmatory results (Section 9) and the exploratory ones (Sections 10 and 11), labelled as such.
- **Part IV.** Recommendations, limitations and how to reproduce every number (Sections 12 to 14). A glossary is in Section A.

Results marked [preregistered] test hypotheses fixed in advance on held-out data; results marked [screen] are exploratory. Every figure has a “How to read this figure” note.

Part I

Fast judges: constructed and real

2 Fast judges on constructed cases

Routing an agent starts with a small decision: is this step easy enough for a cheaper or faster path? The fast-decisions bundle hands such questions to a *decision judge*, a fast model that answers one multiple-choice question with a probability for each option. If the top option is clear enough, the agent acts on it *automatically*; otherwise it falls back to slow reasoning by the host model. This part summarizes the two studies of those judges that preceded the cost measurement. Their full reports are on the eval/judge-realistic branch (commit c62a39c); every number here is regenerated from that branch’s committed evidence.

Key idea

A judge is useful when it is *right when it acts* and *acts often enough to matter*. The two numbers that capture this are the *wrong-automatic rate* (share of all cases where the judge acted and was wrong) and *coverage* (share of cases where it acted at all). A judge that never acts has zero wrong automatic decisions and is useless; a judge that always acts is fast and dangerous.

2.1 The benchmark

10 judges (a hosted Jev model, two cloud LLMs, GPT-6 Luna and GPT-6.1 Sol, and several local models) answered 90 development cases (a screen) and 63 fresh holdout cases whose analysis was preregistered, 3 times each. The cases are deliberately adversarial: file-reading, search, selection and computer-use questions, including 29 holdout cases with a side effect (such as *Buy now*) or a prompt injection. Every answer was scored under the gate the bundle ships: act only when the top probability is at least 0.90 and leads the runner-up by at least 0.20, within 3 s. Labels were checked by two blind reviewers; their agreement with the final labels was perfect on both splits (Cohen’s $\kappa = 1.00$; Table 1).

Table 1: Blind label audit of the judge benchmark: agreement between the two reviewers and the final labels.

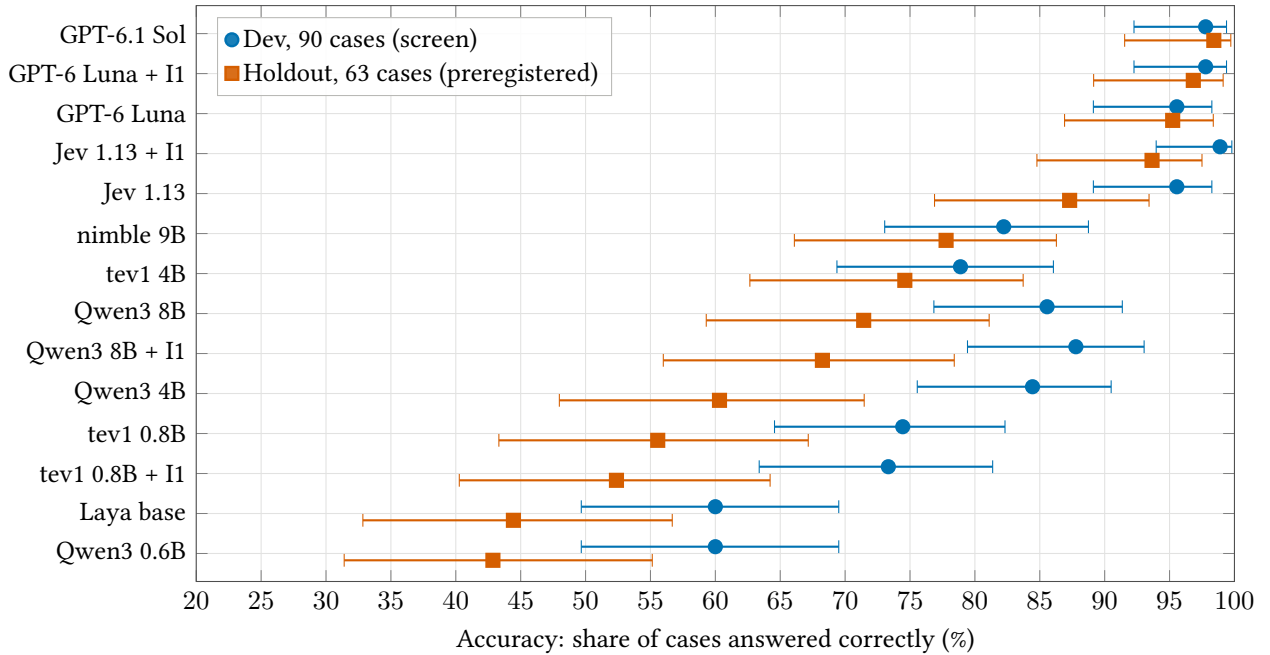
Split	cases	reviewer A vs B	A vs frozen label	B vs frozen label
dev	90	90/90 ($\kappa = 1.00$)	90/90 ($\kappa = 1.00$)	90/90 ($\kappa = 1.00$)
holdout	63	63/63 ($\kappa = 1.00$)	63/63 ($\kappa = 1.00$)	63/63 ($\kappa = 1.00$)

Table 2: Judge benchmark holdout (63 cases, 3 repetitions, the bundle’s gate): accuracy, wrong automatic decisions, coverage, latency and cost per million decisions.

Judge	Accuracy		Wrong automatic		Coverage automatic	Latency (ms)		\$/1M
	correct	95 % CI	count	95 % CI		p50	p95	
GPT-6.1 Sol	62/63	92–100 %	0	0–6 %	47	2,131	4,252	889.37
GPT-6 Luna + I1	61/63	89–99 %	1	0–8 %	49	1,015	1,649	43.37
GPT-6 Luna	60/63	87–98 %	2	1–11 %	51	1,015	1,850	40.46
Jev 1.13 + I1	59/63	85–98 %	1	0–8 %	38	154	242	19.64
Jev 1.13	55/63	77–93 %	1	0–8 %	40	160	222	18.41
nimble 9B	49/63	66–86 %	8	7–23 %	49	221	308	0.00
tev1 4B	47/63	63–84 %	11	10–29 %	43	112	177	0.00
Qwen3 8B	45/63	59–81 %	14	14–34 %	50	138	273	0.00
Qwen3 8B + I1	43/63	56–78 %	14	14–34 %	48	141	231	0.00
Qwen3 4B	38/63	48–71 %	16	16–37 %	38	157	212	0.00
tev1 0.8B	35/63	43–67 %	14	14–34 %	29	40	54	0.00
tev1 0.8B + I1	33/63	40–64 %	14	14–34 %	27	42	369	0.00
Laya base	28/63	33–57 %	12	11–30 %	23	28	42	0.00
Qwen3 0.6B	27/63	31–55 %	16	16–37 %	28	43	54	0.00

2.2 Preregistered holdout result [\[preregistered\]](#)

The default judge, Jev 1.13, was right on 55 of 63 holdout cases (95 % CI 77–93 %) and made 1 wrong automatic decision, at a p95 latency of 222 ms and \$18.41 per million decisions. GPT-6 Luna (60/63) and GPT-6.1 Sol (62/63) were more accurate, but neither difference survives the preregistered Holm correction (adjusted $p = 0.16$ and 0.09), and both fail the latency and cost rules (p95 1,850 and 4,252 ms; \$40.46 and \$889.37 per million). Jev stays the default. No local model qualified as an offline tier. The OpenAI Decisions API returned HTTP 403, so nothing is claimed for it.

**Figure 1:** Judge accuracy with 95 % intervals on the dev screen and the preregistered holdout.

How to read this figure. Each row is a judge; dots are accuracy, whiskers the 95 % interval. Overlapping whiskers among the top three (Jev, Luna, Sol) are why their differences are called “within noise”.

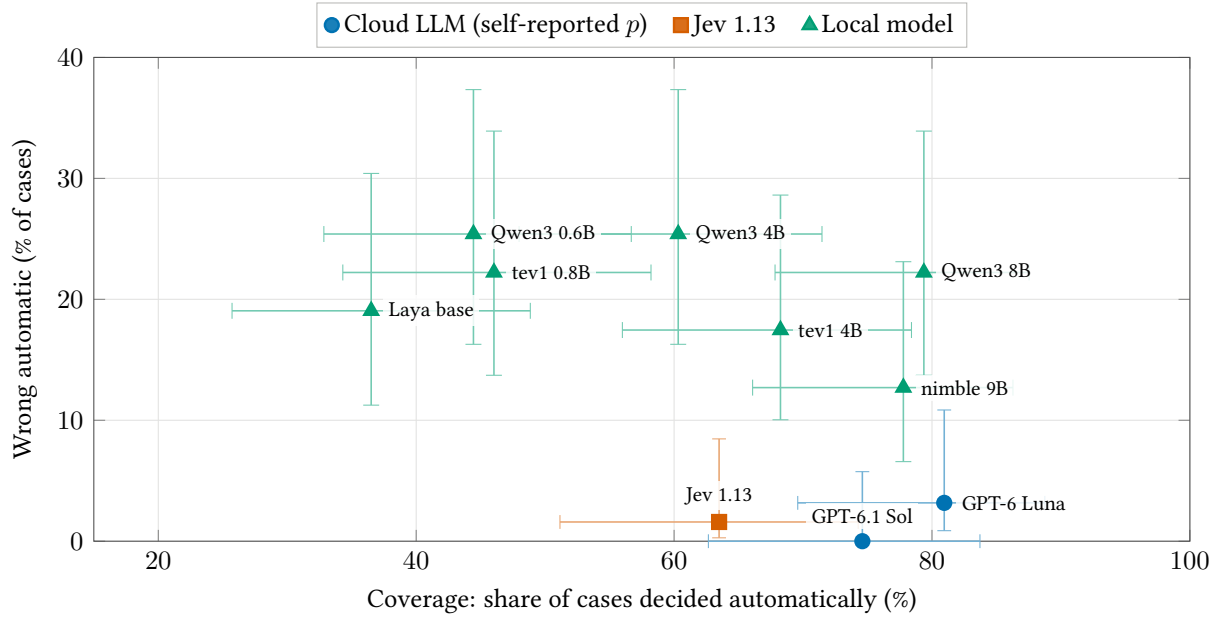


Figure 2: Wrong-automatic rate against coverage on the holdout: the safety–usefulness trade-off of each judge.

How to read this figure. Right means the judge acts more often; up means it is more often wrong when it does. Good judges sit low and to the right. The cloud judges and Jev sit on the floor; the local models are higher.

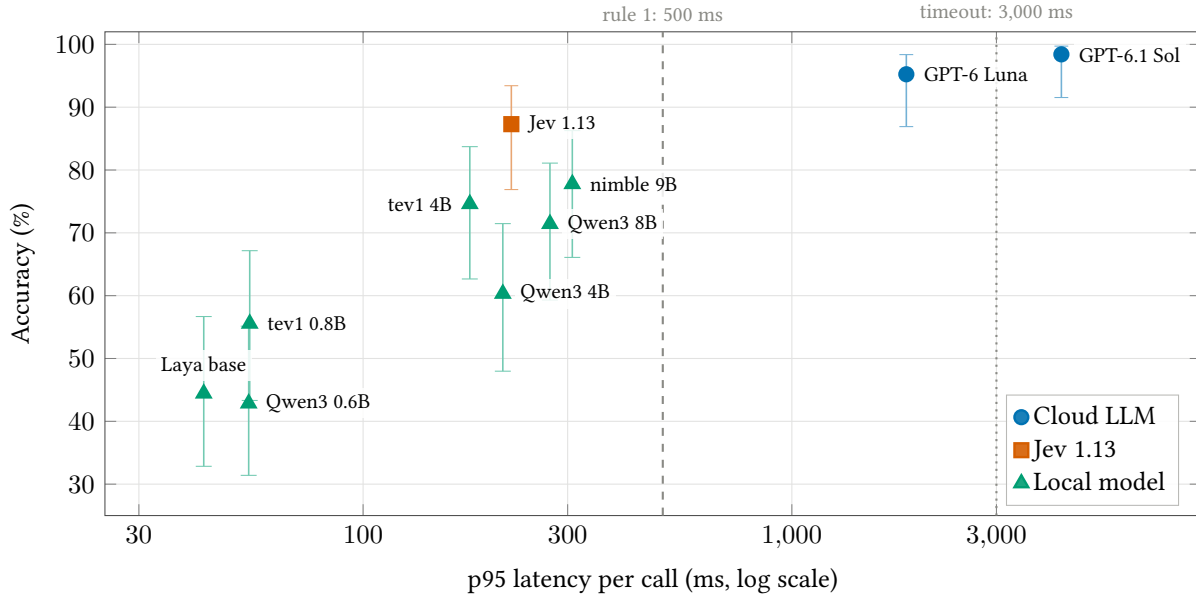


Figure 3: Accuracy against p95 latency (log scale) on the holdout.

How to read this figure. Up is more accurate; left is faster. Jev is the only judge that is both fast and among the most accurate; the cloud LLMs buy a few points of accuracy with seconds of latency.

2.3 Where the time goes, and the side-effect guard

A judge call is mostly waiting. For Jev, about 98 ms of a 150 ms call was network and 52 ms server time; for Luna about 95 % was server time (Fig. 4).

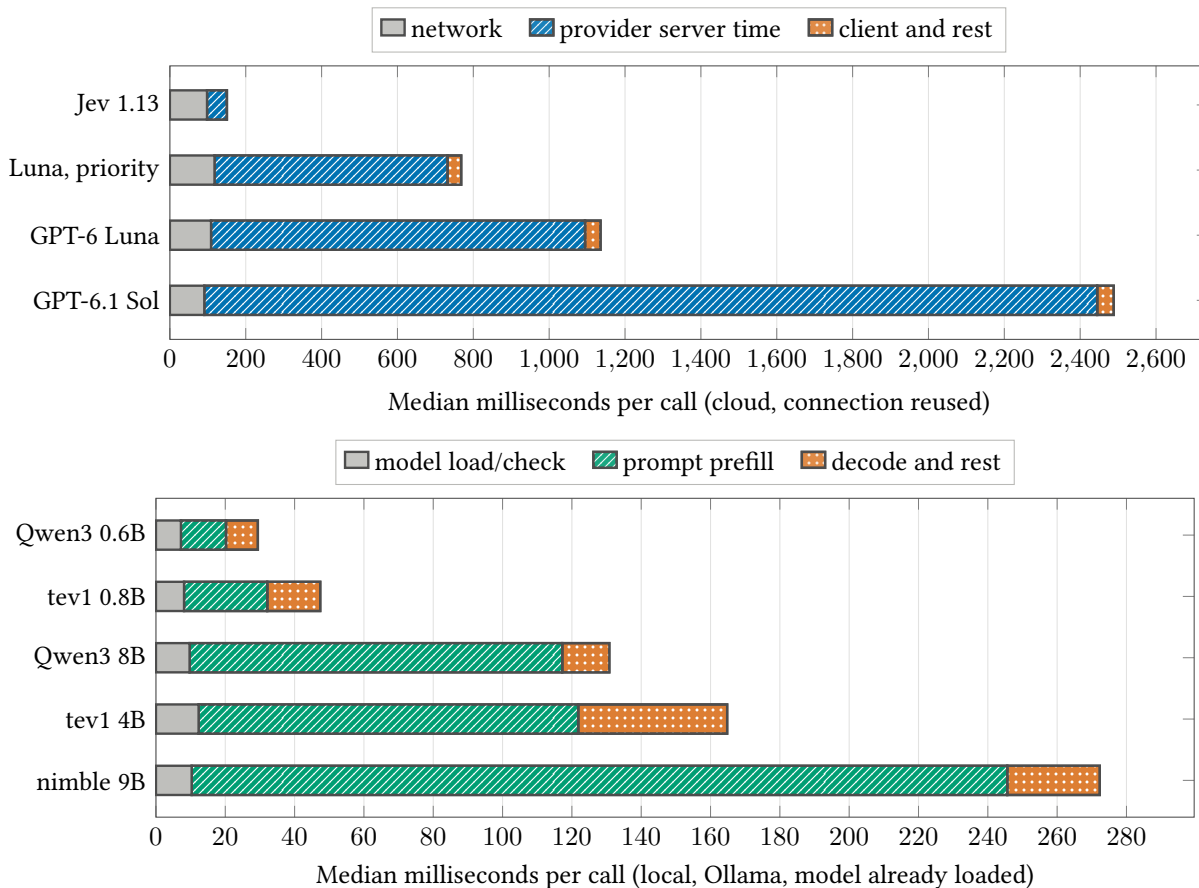


Figure 4: Where the time goes in one judge call: cloud judges (top) and local judges (bottom).

How to read this figure. Each bar is one judge’s median call, split into its parts. Long bars are slow judges; the colour of the longest segment tells you why.

The most dangerous mistakes were confident clicks on side-effect options. A deterministic host guard that refuses to act automatically on an option whose label names an irreversible or external change (intervention I2) removed all 38 such wrong automatic decisions on the holdout, across the 9 judges it applied to, with no change in accuracy (Fig. 5).

Validating an earlier first pass. The benchmark re-examined an earlier, faster comparison: of its 20 claims, 9 were confirmed, 4 refuted, 6 revised and 1 was new. The claim-by-claim history is in Table 28 (appendix).

2.4 Cloudflare Clef judges

The same judge-benchmark splits (dev, holdout) were later run with Cloudflare Clef judges, scored under the bundle’s gate (majority over repetitions). Evidence: 2026-10-04-clef-judges.

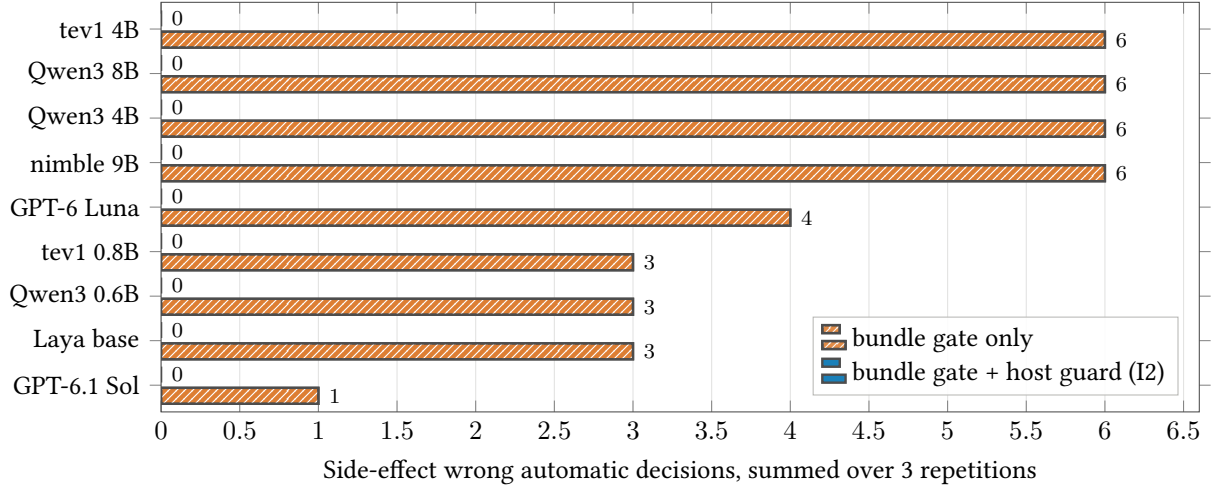


Figure 5: The side-effect guard (I2) on the holdout: targeted wrong automatic decisions before and after.

How to read this figure. Each judge has a “before” and an “after” bar; every “after” bar is zero.

Table 3: Cloudflare Clef judges on the judge-benchmark splits.

Split	Judge	correct	wrong automatic	automatic
dev	clef	85/90	1	69
dev	clef-flash	77/90	7	70
dev	jev-1.13	86/90	2	68
holdout	clef	57/63	3	46
holdout	clef-flash	50/63	6	46
holdout	jev-1.13	56/63	1	40

3 Fast judges on real decisions

Constructed cases can be easier, or harder, than real ones. A second, preregistered study rebuilt 42 real read-shortcut decisions (“read this file, or think?”) from earlier agent sessions, exactly as the in-session judge had seen them, and asked every judge again. Two blind reviewers labelled each case ($\kappa = 0.84$ with each other, but only 0.51–0.57 with what the host actually did next).

Table 4: Real decisions, holdout [preregistered]: 42 cases. “Correct reads” counts correct automatic reads among the 13 cases whose label is a read.

Judge	form	Accuracy		Wrong automatic		automatic	Correct reads (of 13)	Latency (ms)	
		correct	95 % CI	count	upper			p50	p95
Jev 1.13	native	23/42	40–69 %	5	25 %	15	10	163	234
nimble 9B	adapted	13/42	19–46 %	9	36 %	18	9	481	569
GPT-6 Luna	adapted	23/42	40–69 %	5	25 %	13	8	1,280	1,977
GPT-6.1 Sol	adapted	25/42	44–73 %	2	16 %	9	7	3,274	4,975
Qwen3 8B	native	21/42	36–64 %	13	46 %	20	7	241	376
Qwen3 0.6B	native	19/42	31–60 %	9	36 %	15	6	34	46
tev1 4B	adapted	23/42	40–69 %	3	19 %	7	4	255	366
Laya base	native	11/42	15–41 %	0	8 %	0	0	55	66
Qwen3 4B	native	29/42	54–81 %	0	8 %	0	0	141	231
tev1 0.8B	adapted	25/42	44–73 %	0	8 %	0	0	80	98
<i>Always fall back</i>	—	29/42	54–81 %	0	—	0	0	—	—

No judge paid off. The preregistered usefulness rule asked for at least 4 correct automatic reads with a wrong-read upper bound below 10 %; 0 of 10 judges met it. Jev took the most sufficient reads (10 of 13) but made 5 wrong automatic reads; Jev, Luna and Sol were within noise (Holm $p = 1.00$ for Jev against Luna).

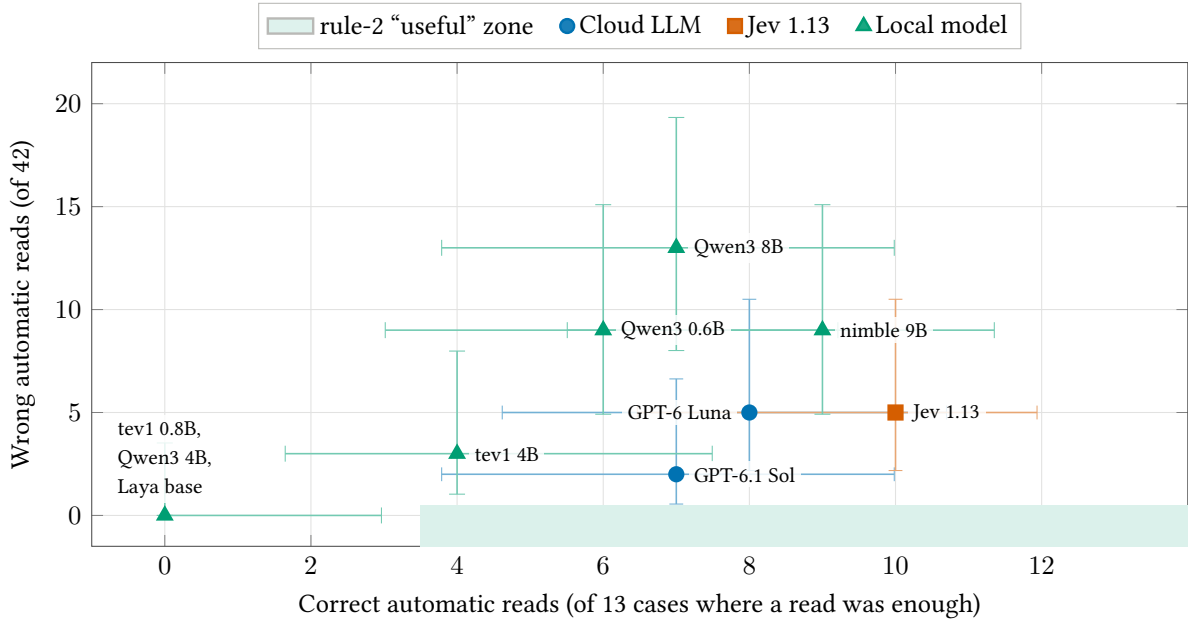


Figure 6: Real decisions: correct automatic reads against wrong automatic reads, holdout [preregistered], with 95 % Wilson intervals. The shaded strip is where the preregistered rule would call a judge useful.

How to read this figure. Right is more savings; up is more misfires. No judge reaches the shaded strip: every judge with enough correct reads also misfired.

Read this before generalizing

- **Selection.** Every case is one an in-session judge had already declined. Jev made 0 wrong reads on the 19 cases from its own sessions and 5 on the 23 others.
- **Label overrides.** Reviewers overrode the host-derived label in 19 cases, leaning strict.
- **Task clipping.** The judge saw a 2,048-character state; in SWE-bench cases the task text ended after about 746 characters, before the issue.
- **An artifact.** Qwen3 4B never answered (126 answers turned into a fallback).

3.1 Four bundle defects, and how they were fixed

Building the real cases exposed four defects. Commit a15c439 (2026-09-30; 9 files, with 4 new test files), revised after review in 1f3b0e4 and merged as PR #58, fixed all four. Earlier, PR #56 (commit 180f919) had tightened the computer-use gate and added the side-effect guard to the computer-use judge. Results in this report were measured as stated in each chapter.

Table 5: The four defects found by the real-decision study and their fixes (PR #58).

Defect	Effect	Fix
State budget clipped the task	The judge saw the operating rules but not the SWE-bench issue	Over-budget task text keeps head and tail within a 2000-character budget and drops leading system-reminder blocks
Escalation and phase judges read the wrong text	Their “task” was the injected system-reminder block	The judge state takes the current turn’s task, clipped the same way
Ollama rejected object criteria	HTTP 400 on read-shortcut questions	A criteria-format option, with a one-time fallback to string criteria
Qwen3 4B never answered on the candidate path	First token was prose, so every answer became a fallback	Retry with an “Answer:” prefill; raise an error instead of a fake confident abstain

Part II

What caching does to savings

4 Why estimates were not enough

Receipts are claims, not measurements. The bundle records an *efficiency receipt* each time it believes it saved money: “this request went to Sonnet, so it cost \$x less.” A receipt is the policy grading its own homework. It assumes that nothing else about the session changes, which is exactly what caching breaks. An earlier survey of 903 routed benchmark pairs found that 0 of them carried receipts at all, so there was nothing to check them against. Where the same survey could compare receipts with measurements (35 pairs of a different, same-model feature), receipts claimed \$2.22 saved while the measured saving was \$−2.80.

Caching makes cost depend on history. In a long conversation most of each request is a re-sent history, and that history is cheap only while it stays in the cache of the model being used (Section 6). A router that moves a conversation between models therefore changes not only the price per token but also how much of

each request has to be paid at the expensive “write” rate. Whether the cheaper model still wins depends on how long the session is, how often it switches, how long the user pauses, and how many more requests the cheaper model makes to do the same work. None of these can be read off a price list.

The reviewer’s question. Reviewing an earlier study, a reviewer put it plainly (personal communication; the comment is recorded in that study’s caching README): for longer chats you have to be confident in the savings for the smaller model. That earlier study re-analysed existing runs. On an Opus host, the shipped router went from $0.95\times$ the plain host’s cost in single-turn sessions to $1.31\times$ in 4-turn sessions; on a Fable host, from $0.49\times$ to $0.62\times$. But those sessions were short, synthetic, partly used for tuning, and the switch itself was never isolated: no arm “decided once and never switched.” It proposed a direct experiment. This report is that experiment.

Key idea

The only way to know what routing costs is to run the same work twice, once routed and once not, under identical conditions, and compare the bills. Everything else in this report is about making “identical” true and checking that it was.

5 What an earlier caching survey showed

Before any new sessions ran, an earlier study re-read 2,240 existing benchmark runs and formed 1,797 matched pairs (903 of them routed): a routed run and a plain run of the same task, campaign and repetition on the same host. It made no model calls. Its question was the reviewer’s: does the routing saving survive longer sessions?

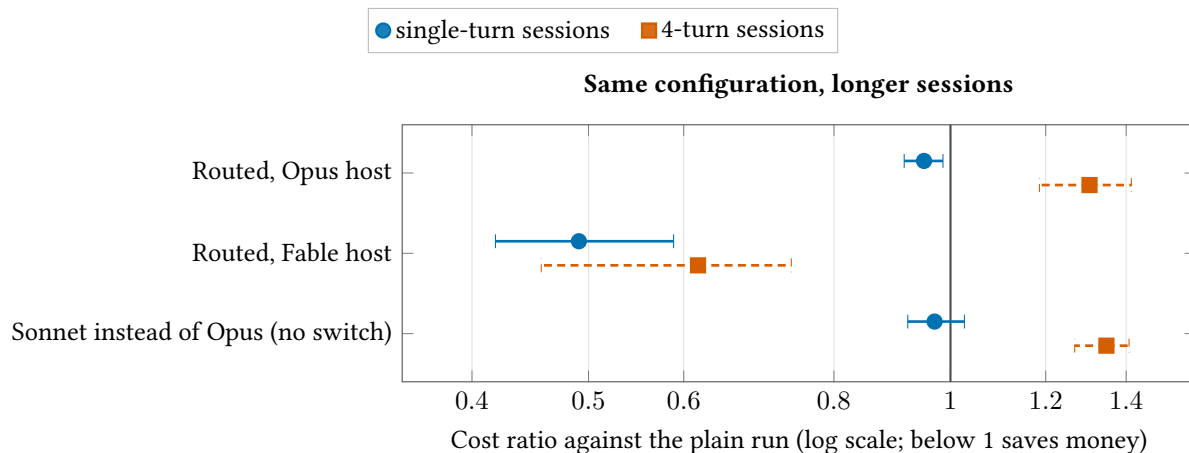


Figure 7: The earlier caching survey: the same configuration in single-turn and 4-turn sessions, with 95 % bootstrap intervals. The top row never switches models: it just runs Sonnet instead of Opus.

How to read this figure. Compare the circle and the square in each row. On the Opus host, routing went from a small saving ($0.95\times$) to a loss ($1.31\times$); the row that never switches moved almost the same way ($0.97\times$ to $1.35\times$). So most of the loss was not the switch itself.

Price and volume, about half each. Re-pricing the control’s identical tokens at Sonnet’s prices gives $0.87\times$ Opus in single-turn sessions but $1.01\times$ in 4-turn ones, because long sessions are mostly cache reads and Sonnet reads cost $1.5\times$ Opus’s while its other token classes cost $0.75\times$. Sonnet also used more requests (volume factor $1.12\times$ to $1.34\times$). Of the rise, about 46 % was the price edge eroding and 54 % extra volume.

Table 6: Earlier survey: share of the routed runs’ cost spent re-writing caches after a model switch.

Kind of session	pairs	sessions with a switch	rebuild share
Single-turn routing	667	1	0.3 %
4-turn, Opus host	90	29	11 %
4-turn, Fable host	75	33	27 %
SWE-bench, mid-turn escalation	11	10	27 %
SWE-bench, turn-start routers	60	17	13 %

Why a new experiment was needed

The survey’s sessions had at most 4 turns, only 6 synthetic scenarios, turns seconds apart (so caches never expired), and no arm that “decides once and never switches”. 0 of the 903 routed pairs carried efficiency receipts. It proposed a direct, preregistered experiment; Part III is that experiment, larger than proposed.

6 How prompt caching works, and what a probe showed

The idea. Every request an agent sends repeats the whole conversation so far: the tool definitions, the system prompt, and every earlier turn. Re-processing all of it each time would be slow and expensive, so the provider keeps a *prompt cache*. The first time a prefix is sent it is *written* to the cache, at a premium. Later requests that start with the same prefix *read* it back at a fraction of the normal price. [Table 7](#) gives the prices used throughout this report.

Table 7: Price per million tokens (US dollars), by model and token class. The same table priced every request in the campaign and in the probe.

Model	uncached input	cache read	cache write	output
Fable 5.1 (host)	10.00	0.25	12.50	50.00
Opus 5.5 (host)	4.00	0.20	5.00	20.00
Sonnet 5 (cheaper model)	3.00	0.30	3.75	15.00

Two things in this table drive the whole report. Fable is expensive on every class, so moving work to Sonnet can save a lot. Opus is cheap to *read* from cache: Sonnet reads cost $1.5\times$ Opus reads, while Sonnet writes cost only $0.75\times$ Opus writes. In a long conversation that is mostly cache reads, Sonnet can cost more than Opus for the same tokens.

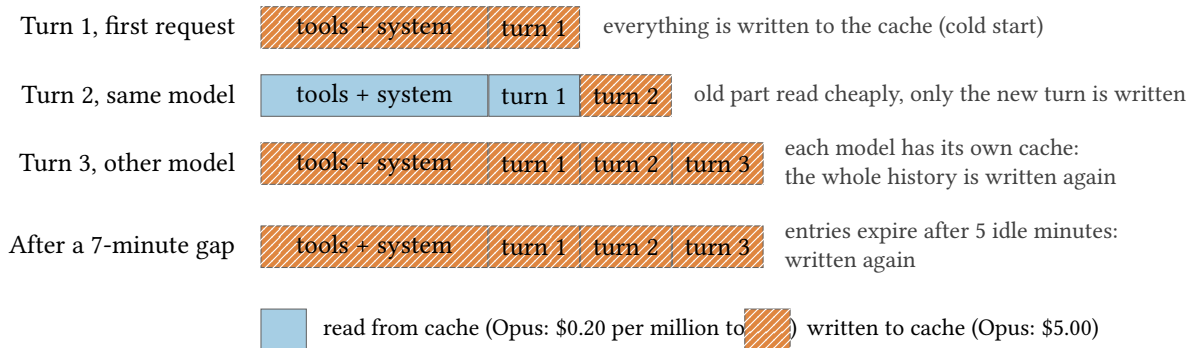
Where the prices come from, and how much the Opus result depends on them. The table is the bundle’s own price table (`savings.DEFAULT_RATES`), last verified against the provider’s public price list on 2026-06-10. The provider’s per-request cost field is computed from the same table, so the exact match between our recomputed costs and the provider’s ([Section 7](#)) checks that tokens were counted correctly, not that the prices are right. The Opus result rests mainly on one number, Opus’s cache-read price. [Table 8](#) re-prices the recorded tokens of the test split, changing only that rate and keeping everything else, including every token count, as measured.

How to read this table. The first row reproduces the confirmatory ratios. Reading down a column shows how the extra cost of routing shrinks as Opus reads get dearer. Sticky reaches parity (ratio 1) at about \$0.38 per million tokens in the pair reading and \$0.34 in the arm reading, roughly $1.90\times$ the table price; shipped at \$0.49 and plain Sonnet at \$0.50 (pair reading). On the raw cost basis with the arm reading the same exercise gives sticky 1.04, 0.98 and 0.93 at \$0.30, \$0.35 and \$0.40.

Table 8: Opus-host cost ratios (test split, tools-normalized) if Opus cache reads cost more than the table’s \$0.20 per million tokens. Only that rate changes; every token count is as recorded.

Opus cache-read price (per M)	Pair reading (primary)			Arm reading		
	sticky	shipped	sonnet	sticky	shipped	sonnet
\$0.20 (table)	1.25	1.38	1.43	1.20	1.32	1.46
\$0.25	1.17	1.29	1.33	1.12	1.24	1.36
\$0.30	1.09	1.22	1.25	1.05	1.16	1.27
\$0.35	1.03	1.15	1.17	0.99	1.10	1.19
\$0.40	0.98	1.09	1.11	0.94	1.04	1.13

So the Opus conclusion is a statement about Opus 5.5 *at these prices*: if its cache reads cost about $1.90\times$ as much, routing sticky to Sonnet would roughly break even on this workload.

**Figure 8:** How the prompt cache behaves over a conversation (schematic, not to scale).

How to read this figure. Each bar is one request; it always re-sends everything before it. Blue parts are read back from the cache at the cheap rate; hatched orange parts are written at the expensive rate. Within one model only the new turn is written. A switch to another model, or a pause longer than the cache’s lifetime, makes the whole history expensive again.

What a probe of the provider showed. Before designing the campaign we sent 111 deliberately constructed requests straight to the provider (cost \$4.76) to pin down the rules that the design depends on. Table 9 shows the evidence; each row quotes the tokens written and read by one request.

Key idea

Six facts from the probe shaped the experiment:

1. **A cache entry exists only where a request marked one, for exactly that prefix.** So one unique string at the very start of a session’s system prompt (a *nonce*) makes every cache entry of that session private to it.
2. **Each model has its own cache**, and on Sonnet each effort level has its own. A switch starts cold, whatever the timing: that cold start is the switching cost we want to measure.
3. **Two API keys do not share a cache**, but there were not enough keys for one per arm.
4. **Two identical requests sent at the same instant both pay the write.** The nonce avoids this too, because no two sessions ever send identical requests.
5. **Every read keeps an entry alive.** Staggering the arms in time would not isolate them, because a running session keeps its shared prefix warm.
6. **Entries expire after 5 idle minutes.** Long pauses in a conversation are a real cost and must be part of the scenarios.

Table 9: The cache-semantics probe. “P” is a prompt prefix of 10,948 tokens; “P+X” a longer prompt of about 15,455 tokens; “Y” a further 2,272-token extension.

Question	What was sent	tokens written	tokens read
Longer entry does not serve its own prefix	Write prompt P+X, then ask for P alone	10,948	0
	Ask for P alone again	0	10,948
A growing conversation reuses the earlier entry	Write P+X, then send P+X+Y	2,272	15,455
Two API keys do not share	Write with key 1, same request with key 2	15,455	0
Simultaneous identical requests both pay	Two at the same instant (each)	15,455	0
Reads keep an entry alive	Read at 4 min, then again at 7 min	0	15,455
	Never read, asked again at 7 min	15,455	0
Each model has its own cache	Written on Sonnet, then sent to Opus	15,454	0
Sonnet caches per effort level	Effort medium written, then no effort	15,464	0
	Same pair on Opus	0	15,464

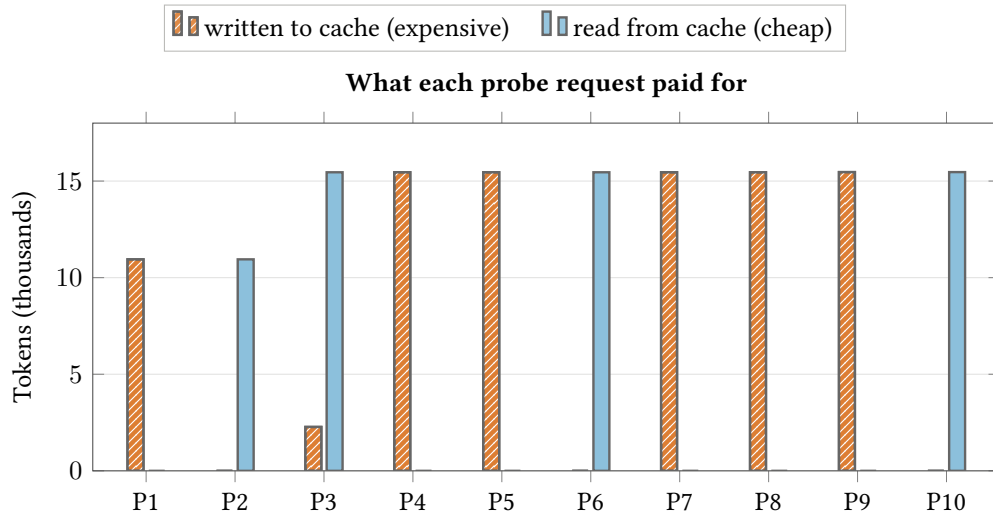


Figure 9: The probe requests of Table 9, drawn: tokens written and read by each.

How to read this figure. A tall hatched bar means the request paid to write its prompt into the cache; a tall blue bar means it found the prompt already cached. Requests that “should” have hit but show only a write (other key, other model, other effort level, an expired entry) are the rules the experiment had to respect. Key: **P1** Write prompt P+X, then ask for P alone; **P2** Ask for P alone again; **P3** Write P+X, then send P+X+Y; **P4** Write with key 1, same request with key 2; **P5** Two at the same instant (each); **P6** Read at 4 min, then again at 7 min; **P7** Never read, asked again at 7 min; **P8** Written on Sonnet, then sent to Opus; **P9** Effort medium written, then no effort; **P10** Same pair on Opus.

Part III

Measuring savings

7 Design of the experiment

7.1 How the design evolved

The first design (v1, docs/design/parallel-measurement-mode-v1.md) wanted to measure the path not taken *inside* a live session: at a decision point, fork the session, run the other path on the same state, record its cost, and throw the result away. It proposed single-call forks for prepared actions, turn forks for routing, and a deferred launch with a “tail correction” to undo cache effects. A pre-flight review rejected three parts of it, and the second design (v2, docs/design/parallel-measurement-mode.md) is what ran:

- **Single-call forks are not a savings unit.** The probe showed that when a call is skipped, the cache write it would have made simply moves to the next call ([Section 6](#)); a one-call fork only re-derives the estimate it was meant to check.
- **Turn forks measure the wrong thing.** A fork that switches model in the middle of someone else’s history measures a switch from a foreign history, not what the policy costs in its own world.
- **Power is counted in scenarios.** Cost ratios vary far more between scenarios than within one, so the unit of replication had to be the scenario ([Section 7.8](#)).

v2 replaced forks with whole paired sessions, isolated by a per-session nonce rather than by timing, and treats cache state as an outcome to be measured, not a nuisance to be corrected.

7.2 Paired sessions from one frozen snapshot

The unit of comparison is a *pair*: one session of a routed arm and one session of the plain host (the *anchor*) on the same scenario, repetition and host. Both start from the *same* frozen copy of the workspace and receive the *same* scripted user messages, so any difference in their bills comes from the policy, not from the task.

All arms of one scenario, repetition and host form a *wave*. A wave’s sessions are launched within seconds of each other, each in its own terminal, so they share the time of day and the provider’s load. The launch order was fixed, not randomised: measured from the first session of its wave, the anchor started on average 1.2 s later, the A/A copy 2.4 s, shipped 2.7 s (at most 26.5 s), sticky 3.8 s and Sonnet 5.1 s. A small systematic effect of launch order cannot be ruled out ([Section 10.4](#)). Each session carries its own nonce, so no arm can read another arm’s cache ([Section 6](#)). A per-request audit checked every session for cache reads it could not have written itself; 0 sessions were flagged.

Key idea

Pairing removes most of the noise. Two scenarios can differ in cost by a factor of ten; two arms on the same scenario differ far less. By comparing each routed session only with its own anchor, the analysis looks at the ratio within a pair, and the big differences between scenarios cancel out.

Table 10: The five arms. Every arm except the Sonnet control runs on both host models.

Arm	What it is, and what comparing it with the anchor tells us
anchor	The plain host model, no routing. The baseline every other arm is compared with.
aa	A second, identical anchor on a seeded 20 % subsample. Its ratio to the anchor should be 1: its spread is the run-to-run noise floor.
shipped	The fast-decisions bundle as shipped (orch-default): its judge re-chooses the model (Sonnet or the host) at the start of each turn; it never switched in the middle of a turn (0 mid-turn switches in 280 sessions).
sticky	Uses the same routing judge, but only once, on the first prompt; the session then stays on the chosen model and never switches. Shipped versus sticky isolates the cost of switching.
sonnet	Plain Sonnet for the whole session (“just use the cheap model”). It does not depend on the host, so one run serves as the control for both hosts.

7.3 Arms and hosts

The two hosts are Fable 5.1, an expensive model (Table 7), and Opus 5.5, a cheaper one with very cheap cache reads. The sticky judge chose Sonnet for 248 of 280 sticky sessions (124 of 140 per host, 89 %) and the host for the rest, so in practice the sticky arm is mostly “Sonnet for the whole session”. The judge’s own calls are not part of any session’s cost: the sticky decision was made once by the harness, and the shipped arm’s per-turn decisions went to a separate judge service (Jev). An earlier benchmark measured Jev at \$46.10 per million decisions on comparable real states; at one decision per turn that adds about 0.05 cents to a shipped session, 0.010 % of a plain Fable session’s cost. That estimate comes from another study and is not measured here.

7.4 Scenarios

There are 70 scenarios in 4 families: 20 coding exercises from a multi-language benchmark (polyglot), 16 bug fixes and features in small public repositories (repos), 20 mixed coding-and-analysis tasks (mixed), and 14 documentation, explanation and review tasks (knowledge). Each is a fixed script of 8 to 16 user turns (11.2 on average), identical for every arm, with automatic checks after each turn and hidden tests at the end. Turns are 10 s apart, except that 23 scenarios contain 2 pauses of 7 minutes, longer than the cache’s 5-minute lifetime, so expiry is part of the workload. The full list is in Table 27.

The preregistered split. Before any session ran, the scenarios were split with a fixed random seed (20261002) into 47 *training* scenarios and 23 *test* scenarios, balanced by family and by whether a scenario has long pauses. The savings model is fitted on training scenarios only; every confirmatory claim uses test scenarios only.

Table 11: Scenarios by family and preregistered split.

Family	training	test	total
polyglot	13	7	20
repos	11	5	16
mixed	14	6	20
knowledge	9	5	14
all	47	23	70

7.5 Quality

A routed arm that saves money by doing worse work has not saved anything. Each turn’s checks give a pass or fail; the *turn-pass fraction* is the share of a session’s turns that passed. The hidden tests at the end give a final pass or fail. A savings claim is allowed only where quality is *non-inferior*: the 95 % lower bound of the mean difference in turn-pass fraction (arm minus anchor) must be above -0.05 , that is, at most 5 percentage points worse.

7.6 What a session costs

Every request’s tokens were priced with the fixed table in [Table 7](#); the recomputed cost matched the provider’s reported cost on every session (0 mismatches). One adjustment makes the comparison fair. Every session’s first request writes the shared tool definitions (about 32,100 tokens) to a cold cache, because the nonce stops sessions from sharing. In production those tools would usually be warm. The primary *tools-normalized* cost re-prices that shared prefix as a cache read; it changes the campaign total by \$15.37, and every result is also reported on the raw basis ([Section B](#)).

7.7 Hypotheses and the decision rule

The hypotheses, their thresholds, the train/test split and the decision rule were fixed in the preregistration. Each is tested on the test split only:

- **H1 (Fable host):** sticky and shipped cost less than the anchor (upper end of the interval below 1).
- **H2 (Opus host):** routing to Sonnet does not save: sticky, shipped and Sonnet all have a lower end above 0.95.
- **H3 (both hosts):** sticky costs no more than shipped (upper end of sticky/shipped at most 1.05).
- **Quality:** every savings claim has non-inferior turn-pass fraction.

The decision rule: on a host where H1 holds with non-inferior quality, route (preferring sticky if H3 holds); on a host where H2 holds, do not route to Sonnet. A savings model fitted on training data must also pass a check on the test data: its 90 % prediction intervals must cover between 0.80 and 0.97 of test pairs, and the measured test total must fall inside its predicted interval.

What was fixed later. The *estimator* was not in the preregistration. The confirmatory script, `evals/paired_confirm.py` (commit `c3ea41a`), was written after the data were collected, about 56 minutes after the campaign finished, when exploratory summaries over all scenarios (the savings-model output in `model/`) already existed. It fixed the number of bootstrap resamples (10,000) and the seed (20261002), the pair-level quality filter, which cells count as savings claims, and the verdict rule, including what “contradicted” means. These choices are listed with the other clarifications in [Section B](#); the verdicts under the alternative readings are reported alongside.

Ratios and intervals. The cost of a pair is summarized as a *ratio*, arm cost divided by anchor cost: 0.80 means 20 % cheaper. Ratios are averaged as a *geometric mean*, which treats “twice as expensive” and “half as expensive” symmetrically. A 95 % *confidence interval* gives the range of ratios consistent with the data. It is computed with a *scenario-cluster bootstrap*: resample whole scenarios (then repetitions) with replacement thousands of times, recompute the ratio each time, and take the middle 95 %. Resampling scenarios rather than single sessions keeps the interval honest about how much the conclusions depend on which tasks were chosen.

7.8 How big the experiment had to be

Earlier data gave the spread of log cost ratios between scenarios and within a scenario. Because the between-scenario spread dominates, precision is bought with more *scenarios*, not more repetitions. [Table 12](#) shows the half-widths the design document projected; the campaign used 70 scenarios and 2 repetitions.

Table 12: Projected precision of the routing ratio, from the design document (before the campaign). The projections assumed 60 and 40 scenarios; the confirmatory test split has 23, so its intervals are wider than these.

Contrast (earlier data)	SD of log ratio		95 % CI half-width	
	between scenarios	within	60 scenarios	40 scenarios
Routing, Fable (s1m values)	0.265	0.297	$\pm 9\%$	$\pm 11\%$
Routing, Fable (S1 values, worst case)	0.433	0.258	$\pm 13\%$	$\pm 16\%$
Routing, Opus (S1 values)	0.162	0.266	$\pm 7\%$	$\pm 8\%$
Routing, Opus (s1m values)	0.027	0.211	$\pm 4\%$	$\pm 5\%$

Behind that table is a variance analysis of 1,787 earlier pairs (`step0_variance.json`). For the shipped router on the Fable host in 4-turn sessions, the between-scenario standard deviation of the log ratio was 0.25 against 0.32 within a scenario; on Opus it was 0.07 against 0.17. [Figure 10](#) turns these into the interval one should expect for a given number of scenarios.

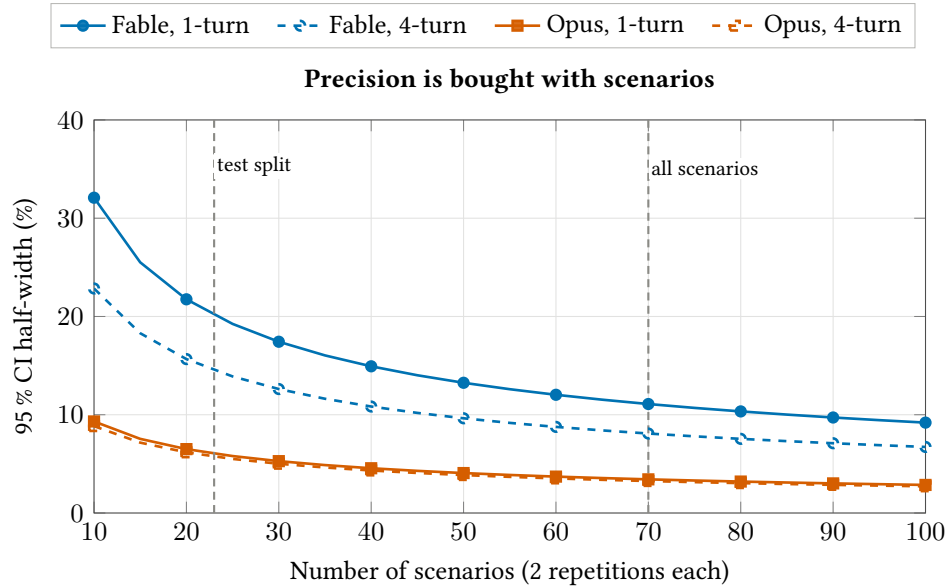


Figure 10: Projected 95 % interval half-width of the cost ratio against the number of scenarios (two repetitions each), from the earlier data's between- and within-scenario variance.

How to read this figure. Lower is more precise. Each line is one earlier configuration (host, session length). The curves fall quickly at first and then flatten: going from 23 to 70 scenarios roughly halves the uncertainty. The Fable single-turn line is high because those ratios varied a lot between tasks.

7.9 The pilot

Two small screening pilots preceded the preregistration; the second ran 45 sessions on 5 scenarios. They confirmed the plumbing (nonce isolation, cache audit, quality checks) and measured the A/A noise: a standard deviation of the log ratio of 0.10 on Fable and 0.18 on Opus. Their cost ratios became the hypotheses above, not findings; they are compared with the final numbers in [Section 10.5](#).

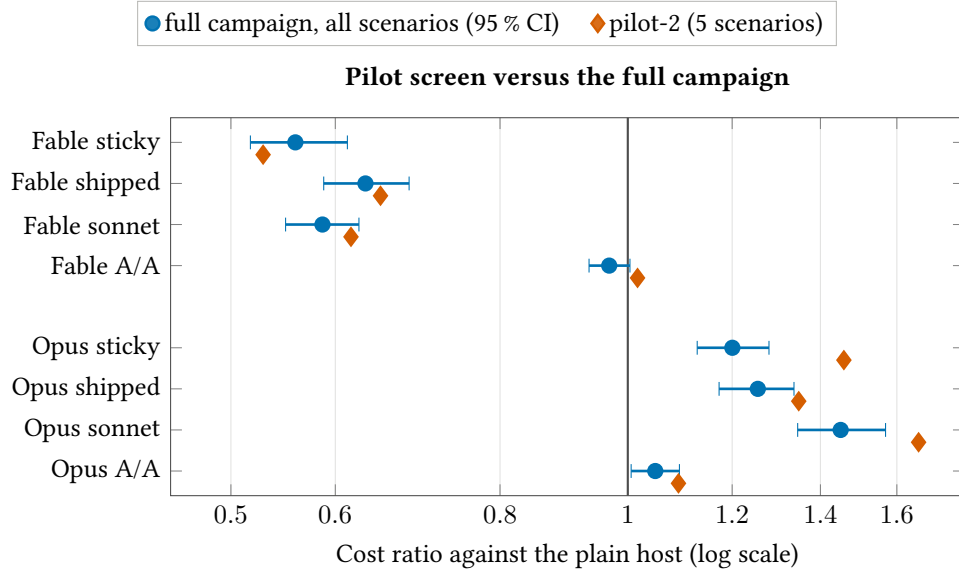


Figure 11: The second screening pilot (40 pairs on 5 scenarios) against the full campaign (all scenarios, 95 % intervals), for each arm and host. The first pilot was a smoke test of the harness (\$43.72) whose rows were not packaged.

How to read this figure. Diamonds are the pilot, dots and whiskers the final result. On Fable the diamonds sit inside or near the whiskers; on Opus the pilot overstated the extra cost. A five-scenario pilot was good for finding the direction, not the size.

8 Running it safely

8.1 The memory guard

On the day the campaign was prepared, a validation run nearly ended it. An unguarded Go test binary from one scenario grew to between 120 and 470 GB of memory and drove the 128 GB workstation out of memory four times, taking the terminal server down with it. A benchmark that occasionally crashes its own machine produces gaps that are not random: the heavy scenarios go missing.

The operating system’s own out-of-memory killer (JetsamEvents in the macOS logs) recorded each of those events. From then on nothing that executes scenario code ran unguarded:

- **memguard** runs every grader and hidden test in its own process group, measures the memory of the whole process tree every 0.25 s, and kills it above 4 GB or after 300 s. Such a kill counts as a failed check, never as a crash.
- A **watchdog** in the campaign runner checks every 1 s. It kills an agent session above 8 GB, pauses new waves when free memory runs low, and below 16 GB kills the newest sessions first.
- **The same limits apply to every arm**, so the guard cannot favour one policy.

Key idea

A guard that kills runaway sessions is only fair if its kills are counted, not hidden. The one session the watchdog killed (py-forth-r2-opus-aa: 9.51 GB against an 8 GB cap) was kept as a quality outcome (turn-pass fraction 0.8) and excluded only from cost comparisons, which is why 895 of the 896 pairs are cost-valid.

8.2 Failures and retries

A *wave attempt* that failed for an infrastructure reason was rerun as a whole, so that all arms of a wave always shared the same conditions. Of 329 wave attempts, 280 were accepted, one per wave, and 49 failed. Every failed attempt was an infrastructure failure, every wave eventually completed, and no wave is missing from the data. 10 waves were excluded after three failures and then reset and rerun by hand.

Table 13: Failed wave attempts by cause. For 8 of them the cause was not recorded at the time and is inferred from timing.

Cause	failed attempts	sessions in them	spend
Terminal-server session cap (harness bug)	31	119	\$0.00
Terminal-server daemon restart	11	37	\$48.51
Laptop lid closed on battery	7	20	\$55.46
Total	49	176	\$103.97

The largest group was a harness bug: after a restart at high parallelism, the runner launched more terminals than the terminal server allowed, and the excess launches failed at once without spending anything. The fix (wait for capacity) went into the running harness that evening. Restarts of the terminal server killed sessions mid-run; a circuit breaker then paused and resumed the campaign. The laptop’s lid being closed on battery put the machine to sleep twice, and those attempts timed out.

8.3 What the live dashboard showed

During the run a local dashboard refreshed every minute with progress, throughput and spend. Its final snapshot is packaged as `dashboard/index.html`. [Figure 12](#) rebuilds the same views from the session rows and the failure log, so it can be checked against the data.

8.4 Spend and time

The campaign ledger recorded \$3,527.29 against a hard stop of \$6,500: \$3,418.15 for the accepted sessions, \$103.97 for failed attempts and \$5.17 for preflight checks. The two pilots cost \$163.26. The first session started at 2026-10-01 14:24 UTC and the campaign completed at 17:39 EDT the next day, about 31 hours.

Provenance. The preregistration was committed at 10:18 EDT (commit 3aa2d1e); the first session started 6 minutes and 12 seconds later. The agent under test was frozen at that same commit in every session. The campaign *harness*, however, ran from uncommitted working-tree code and was committed afterwards (dc1aa4e); re-extracting the rows with the committed harness reproduces them byte for byte ([Section 14](#)).

9 Confirmatory results [\[preregistered\]](#)

Everything in this section uses the 23 test scenarios only (294 pairs). The hypotheses, thresholds and decision rule are the preregistered ones; the estimator (resamples, seed, quality filter, verdict rule) was fixed afterwards in the confirmatory script ([Section 7](#)). The preregistration said the cost ratio is computed “among pairs where quality is non-inferior.” That sentence has two readings, so both are reported:

- **Pair reading (primary):** keep a pair only if its own turn-pass fraction is not worse than its anchor’s by more than 5 points. In this design that keeps pairs where the arm did at least as well.

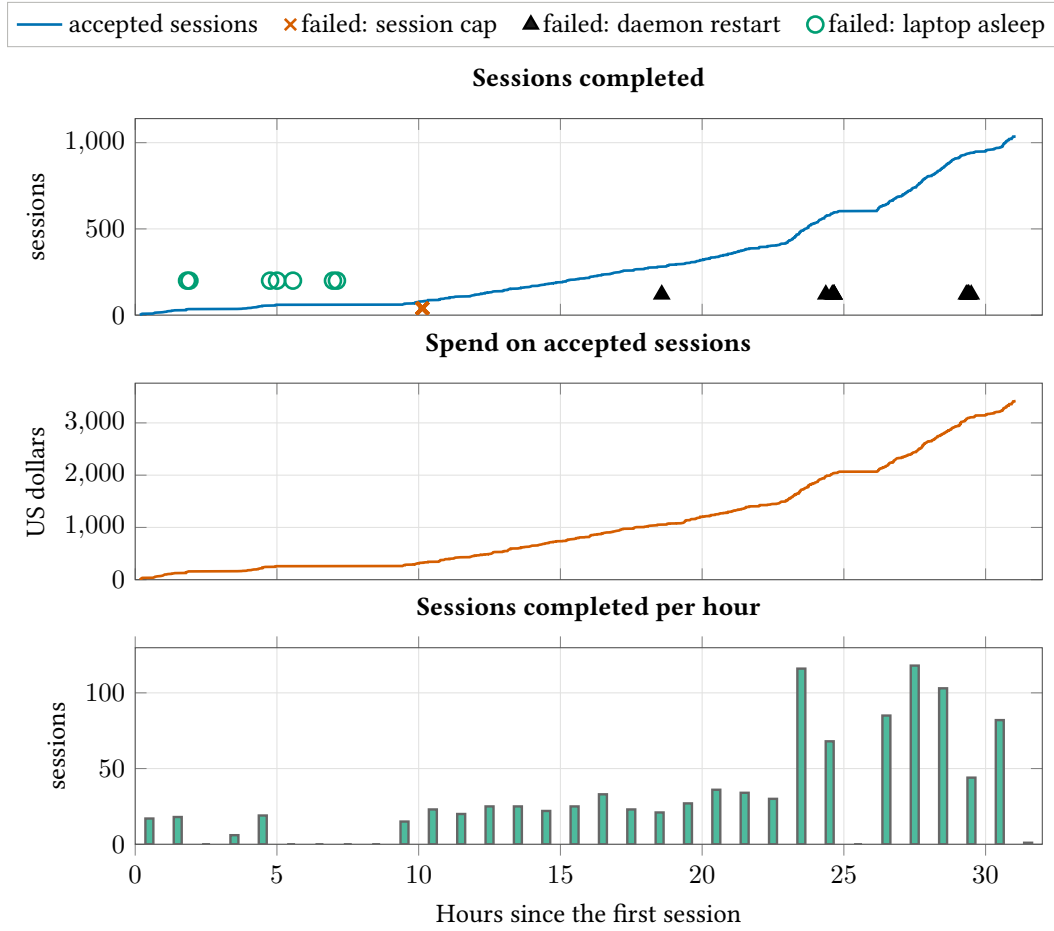


Figure 12: Campaign progress reconstructed from the data: accepted sessions completed and their spend over the 31.1 hours from the first session to the last, sessions completed per hour, and when failed attempts started, by cause.

How to read this figure. Flat stretches in the top two panels are pauses: the laptop asleep, a terminal-server restart, or the runner waiting for capacity. The markers on the top panel show when failed attempts started; the burst of crosses is the session-cap bug, which cost nothing because those launches failed immediately. The bottom panel peaks at 118 sessions in an hour.

- **Arm reading (sensitivity):** keep every cost-valid pair, and require non-inferiority of the arm as a whole.

Table 14: Primary endpoint: geometric-mean cost ratio, arm over plain host, test split, with 95 % scenario-cluster bootstrap intervals. “Pairs kept” counts pairs that pass the pair-level quality filter out of all cost-valid test pairs.

Host	Arm	Pair reading (primary)			Arm reading (sensitivity)	
		pairs kept	ratio	95 % CI	ratio	95 % CI
Fable 5.1	sticky	38/46	0.558	0.49–0.64	0.537	0.48–0.61
Fable 5.1	shipped	39/46	0.629	0.56–0.71	0.618	0.55–0.70
Fable 5.1	sonnet	39/46	0.577	0.52–0.65	0.590	0.53–0.66
Opus 5.5	sticky	38/46	1.249	1.13–1.39	1.203	1.08–1.33
Opus 5.5	shipped	38/46	1.380	1.25–1.53	1.321	1.19–1.46
Opus 5.5	sonnet	40/46	1.433	1.25–1.64	1.457	1.28–1.67

How to read this table. A ratio below 1 is a saving: 0.558 means the routed arm paid about 56 cents for every dollar the plain host paid. If the whole interval is below 1, the cost difference is not explained by chance; if it is above 1, neither is the extra cost. The pair-level filter drops sessions that were cheaper but did worse, so for sticky and shipped it raises the ratio by 0.01 to 0.06; on Opus the arm reading gives 20–46 % more than the plain host.

9.1 The hypotheses

Table 15: The preregistered hypotheses, test split, pair reading. The arm reading gives the same verdicts.

Hypothesis (test split, pair reading)	Host	Arm	ratio	95 % CI	verdict
H1: routing is cheaper (upper end < 1)	Fable	sticky	0.558	0.49–0.64	confirmed
H1: routing is cheaper (upper end < 1)	Fable	shipped	0.629	0.56–0.71	confirmed
H2: no saving (lower end > 0.95)	Opus	sticky	1.249	1.13–1.39	confirmed
	Opus	shipped	1.380	1.25–1.53	confirmed
	Opus	sonnet	1.433	1.25–1.64	confirmed
H3: sticky/shipped (upper end $\leq$ 1.05)	Fable	sticky/shipped	0.862	0.76–0.99	confirmed
	Opus	sticky/shipped	0.917	0.85–0.99	confirmed

- **H1, confirmed.** On the Fable host, sticky cost $0.558 \times$ (95 % CI 0.49–0.64) and shipped $0.629 \times$ (0.56–0.71) of the plain host.
- **H2, confirmed.** On the Opus host, sticky cost $1.249 \times$ (1.13–1.39), shipped $1.380 \times$ (1.25–1.53) and plain Sonnet $1.433 \times$ (1.25–1.64): routing to Sonnet costs about 25–43 % more.
- **H3, confirmed.** The preregistered test was only that sticky costs no more than shipped (upper end at most 1.05). Sticky cost $0.862 \times$ of shipped on Fable (0.76–0.99) and $0.917 \times$ on Opus (0.85–0.99), on 35 and 33 matched scenario repetitions. On Fable the upper end, 0.994, is close to 1.

9.2 Quality

Every Fable savings claim passes the quality test. (A *savings claim* here is any arm and host whose own cost interval shows a saving; only those cells are tested for quality.) The closest call is Fable sticky, whose lower bound is -0.047 , inside the -0.05 margin by only 0.0035 . Across all 70 scenarios (exploratory) its mean difference is -0.021 with a 95 % interval of -0.058 to 0.012 : there the margin is not shown. Opus sticky narrowly fails it (lower bound -0.058), but no savings claim is made on Opus, so the quality hypothesis holds as preregistered.

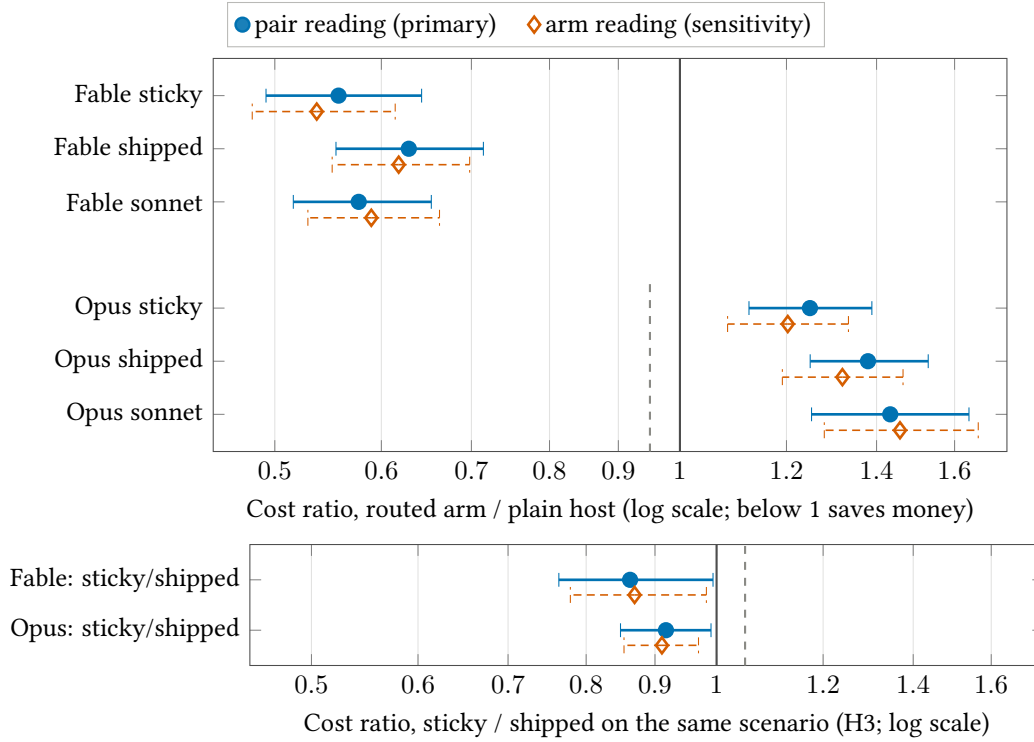


Figure 13: Confirmatory cost ratios on the test split. Top: each routed arm against the plain host. Bottom: the sticky arm against the shipped arm on the same scenarios (H3). Whiskers are 95 % intervals. The solid line marks 1 (no difference); the dashed lines mark the preregistered thresholds for H2 (0.95) and H3 (1.05).

How to read this figure. Each row is one comparison; the dot is the best estimate and the whisker the 95 % interval. The horizontal axis is logarithmic, so “half the cost” and “double the cost” are the same distance from 1. All three Fable rows lie well left of 1 (savings); all three Opus rows lie right of the dashed line (no saving). In the bottom panel both hosts lie left of 1: the sticky policy cost less than the shipped one, by a small margin on Fable.

Table 16: Quality: mean difference in turn-pass fraction, arm minus plain host, test split, all quality-valid pairs. Non-inferior when the lower end of the interval is above -0.05 .

Host	Arm	pairs	mean Δ turn-pass	95 % CI	non-inferior
Fable 5.1	sticky	46	0.002	$-0.047-0.055$	confirmed
Fable 5.1	shipped	46	0.003	$-0.041-0.054$	confirmed
Fable 5.1	sonnet	46	0.009	$-0.036-0.061$	confirmed
Opus 5.5	sticky	46	-0.010	$-0.058-0.033$	not confirmed
Opus 5.5	shipped	46	-0.001	$-0.035-0.037$	confirmed
Opus 5.5	sonnet	46	0.007	$-0.036-0.051$	confirmed

9.3 The savings model

The preregistered deliverable is a model that predicts the saving of a new session from what is known before it starts: arm, host, task type, number of turns, long pauses and the size of the first prompt. It was fitted on the training scenarios only and then asked to predict the 276 held-out test pairs.

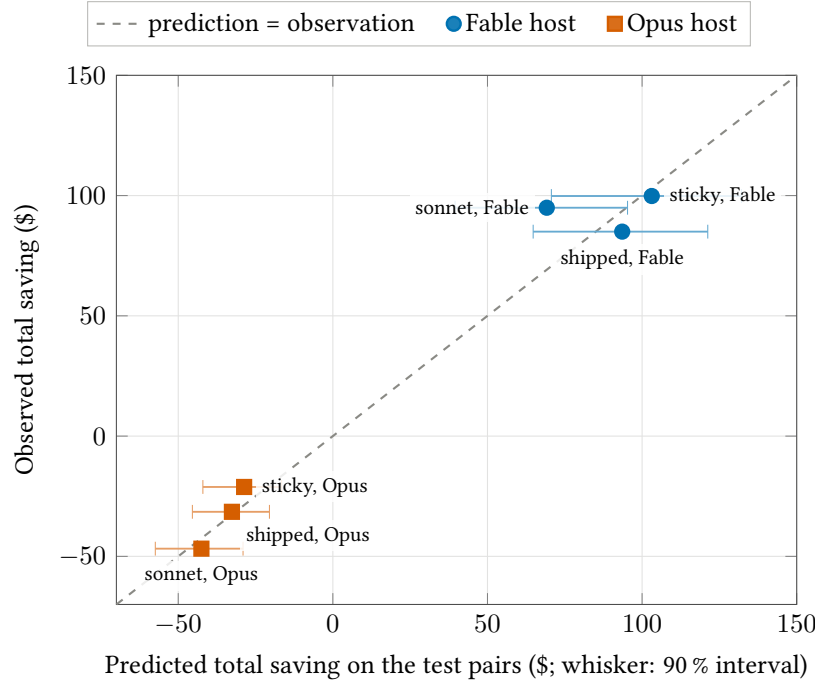


Figure 14: The savings model on the test split: predicted total saving (with its 90 % interval) against the observed total, for each arm and host.

How to read this figure. A point on the dashed diagonal is a perfect prediction. Every whisker crosses the diagonal, so every observed total lies inside its predicted 90 % interval. Points above zero are savings (Fable); points below zero are extra costs (Opus).

The model passed both preregistered checks. Its 90 % intervals covered 0.960 of test pairs on the log ratio and 0.935 on the dollar saving, both inside $[0.80, 0.97]$. The observed test total, \$180.35, lay inside the predicted interval of \$63.20 to \$251.26 (point prediction \$162.04). The intervals are conservative: they covered 0.960 of pairs against a nominal 0.90. The pooled total nets Fable savings (\$279.78) against Opus losses (\$99.43), so it says little about either host on its own. The design document’s stricter criteria (coverage 0.83–0.97, calibration slope 0.7–1.3) are also met (slope 0.83). [Table 21](#) in the appendix gives the check for each arm and host. The verdict is “predictive”; its limits are discussed in [Section 10.6](#).

9.4 Decision

Applying the preregistered rule: on the Fable host, **route (sticky)**; on the Opus host, **do not route to Sonnet; use the plain host model**. Every verdict was re-derived under both non-inferiority readings and under 20 other bootstrap seeds (20261003–20261022); 0 verdicts changed. The other clarifications of the preregistration ([Section B](#)) were not re-run separately.

10 Exploratory results [screen]

Everything in this section was not preregistered, or uses all 70 scenarios (training and test together). It explains and qualifies the confirmatory results; it does not confirm anything on its own.

10.1 Where the cost comes from

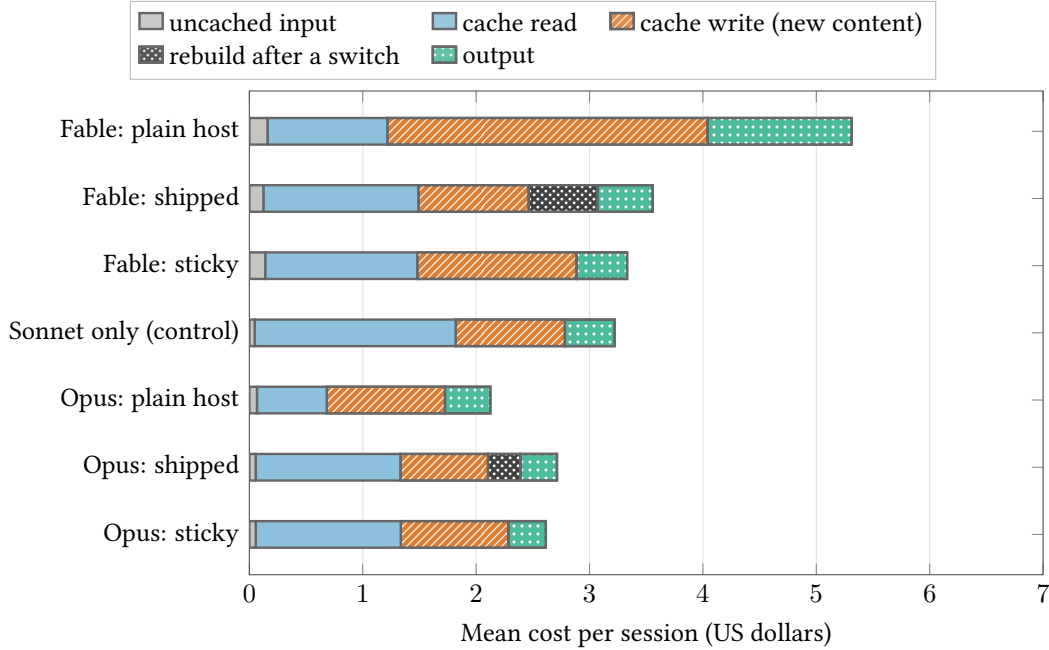


Figure 15: Mean cost per session, split by what was paid for (all splits, cost-valid sessions, tools-normalized). “Rebuild after a switch” is the extra paid for re-writing history to a new model’s cache.

How to read this figure. Each bar is one kind of session; its length is the average bill and its segments show what the bill was for. Compare the three Fable bars with each other, and the three Opus bars with each other. On Fable, routing shrinks the large hatched “cache write” segment. On Opus, routing lengthens the blue “cache read” segment.

Table 17: The numbers behind Fig. 15: dollars per session by token class, main-loop requests per session, and cache-read tokens per session (millions).

Session kind	sessions	\$/session	of which (\$ per session)				requests	tokens read
			cache read	cache write	rebuild	output		
Fable: plain host	140	5.31	1.06	2.82	0.00	1.27	46.0	4.22 M
Fable: shipped	140	3.56	1.37	0.97	0.61	0.49	50.6	4.68 M
Fable: sticky	140	3.33	1.34	1.40	0.00	0.45	50.2	4.58 M
Sonnet only (control)	140	3.22	1.77	0.96	0.00	0.44	57.4	5.90 M
Opus: plain host	140	2.13	0.62	1.04	0.00	0.40	36.4	3.07 M
Opus: shipped	140	2.71	1.28	0.77	0.29	0.32	48.9	4.43 M
Opus: sticky	140	2.61	1.28	0.95	0.00	0.33	48.5	4.43 M

Figure 15 and Table 17 show why the two hosts disagree.

- **On Fable, routing cuts the most expensive line.** A plain Fable session paid \$2.82 of its \$5.31 for cache writes (53 %), and a Sonnet cache write costs $0.3\times$ a Fable one. Moving the session to Sonnet cuts that line by more than the extra reads add.

- **On Opus, routing makes the session longer and its reads dearer.** A sticky session on the Opus host made 48.5 requests against the plain host’s 36.4 ($1.33\times$), read 4.43 million cached tokens against 3.07 million, and paid $1.5\times$ Opus’s price for each read: cache reads rose from \$0.62 to \$1.28 per session.
- **Switching adds a rebuild.** The shipped arm re-chooses its model at the start of each turn and switched models 1.51 times per session on average, always between turns, in 57 % of Fable sessions; rebuilds were 17 % of its cost on Fable and 11 % on Opus. Sticky never switches (0 switches per session), so it never pays this line.

10.2 By task type, pause pattern and length

Table 18: Measured saving per 1,000 sessions (US dollars) by task type, host and arm, all splits. Ranges are in Table 26.

Task type	scenarios	Fable host: saving per 1,000 sessions (\$)			Opus host		
		sticky	shipped	sonnet	sticky	shipped	sonnet
bugfix	18	1,346	797	871	−1,037	−1,240	−1,948
feature	13	2,183	2,019	2,895	−574	−490	−1,099
mixed	28	2,020	1,941	2,405	−303	−401	−815
docs	4	2,615	2,263	2,246	264	162	−129
explain	3	2,315	2,244	2,367	341	235	242
review	4	3,007	3,011	2,389	−425	−642	−1,193
all tasks	70	1,980	1,754	2,090	−489	−588	−1,096

Across all tasks, per 1,000 sessions, sticky saved \$1,980 on Fable (90 % range \$1,577–\$2,368), shipped \$1,754 and Sonnet \$2,090. On Opus the same arms *lost* \$−489, \$−588 and \$−1,096. These dollar figures assume this campaign’s mix of tasks and session lengths.

Long pauses and session length move the ratios less than the host does (Tables 24 and 25 in the appendix). On Opus, routed sessions with two long pauses came closer to break-even (shipped $1.17\times$ against $1.30\times$ without), plausibly because the plain host also has to re-write its expired cache after a pause. On Fable the ratios hardly move with pauses or length.

10.3 Final hidden tests: a caution

Table 19: Final hidden-test pass rate by arm, against the plain host on the same pairs, all splits. McNemar’s test compares the scenarios only the anchor passed with those only the arm passed.

Host	Arm	pairs	arm pass	anchor pass	anchor-only / arm-only	McNemar p
Fable 5.1	sticky	140	0.943	0.979	6 / 1	0.13
Fable 5.1	shipped	140	0.936	0.979	7 / 1	0.07
Fable 5.1	sonnet	140	0.943	0.979	5 / 0	0.06
Fable 5.1	second anchor (A/A)	28	0.964	1.000	1 / 0	1.00
Opus 5.5	sticky	140	0.936	0.957	5 / 2	0.45
Opus 5.5	shipped	140	0.936	0.957	4 / 1	0.38
Opus 5.5	sonnet	140	0.943	0.957	4 / 2	0.69
Opus 5.5	second anchor (A/A)	28	0.929	1.000	2 / 0	0.50

The turn-by-turn quality test is the preregistered one, and it passes. The final hidden tests tell a slightly less comfortable story. On Fable, each routed arm lost more scenarios that the anchor passed than it won: shipped 7 to 1, sticky 6 to 1, Sonnet 5 to 0. McNemar’s test asks whether such a lopsided split could be chance; here p

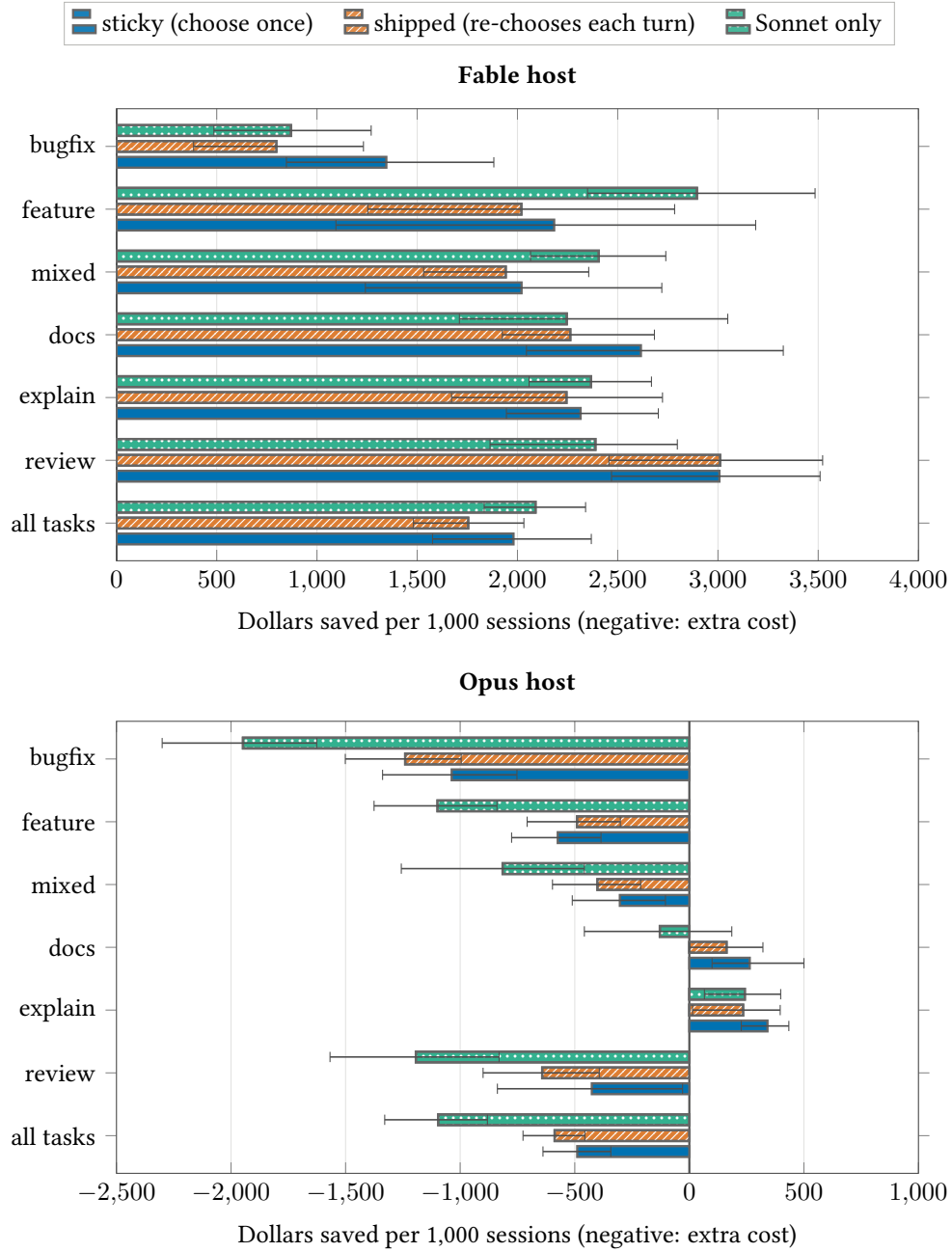


Figure 16: Dollars saved per 1,000 sessions by task type and arm, measured on all 70 scenarios, with 90 % scenario-bootstrap ranges. The two panels have different scales.

How to read this figure. Bars to the right of the zero line are savings; bars to the left are extra costs. On Fable every bar is to the right. On Opus most bars are to the left, except for the short read-heavy task types (docs, explain), which are small and rest on 3 and 4 scenarios: treat them as hints, not findings.

ranges from 0.06 to 0.13, so it could. With so few discordant scenarios the test has little power. Pooling the three routed Fable arms gives a clearer picture: they passed the final hidden tests 3.8 points less often than the plain host across all scenarios (95 % interval -7.9 to -0.5 points) and 2.2 points less often on the test scenarios (-6.5 to 0.0). This is exploratory and was not preregistered, but it is a real caution: a routed deployment should watch the final pass rate.

10.4 The noise floor

The A/A arm runs the plain host twice on the same scenario. Its ratio should be 1, and its spread says how much two identical sessions differ by chance. The standard deviation of the A/A log ratio was about 0.11 per pair, or 0.08 per session, far smaller than the effects above (Table 22). On Fable the A/A ratio was 0.968 (0.94–1.00). On Opus it was 1.049 (1.01–1.09): the interval excludes 1. Two identical Opus sessions differed by about 5 % on average, plausibly because the launch order was fixed (Section 7). If every Opus arm carried the same bias, removing it would move the Opus ratios from 1.25–1.43 to about 1.19–1.37, with the lowest lower bound at 1.07; the H2 verdicts would hold.

10.5 Interim numbers versus final

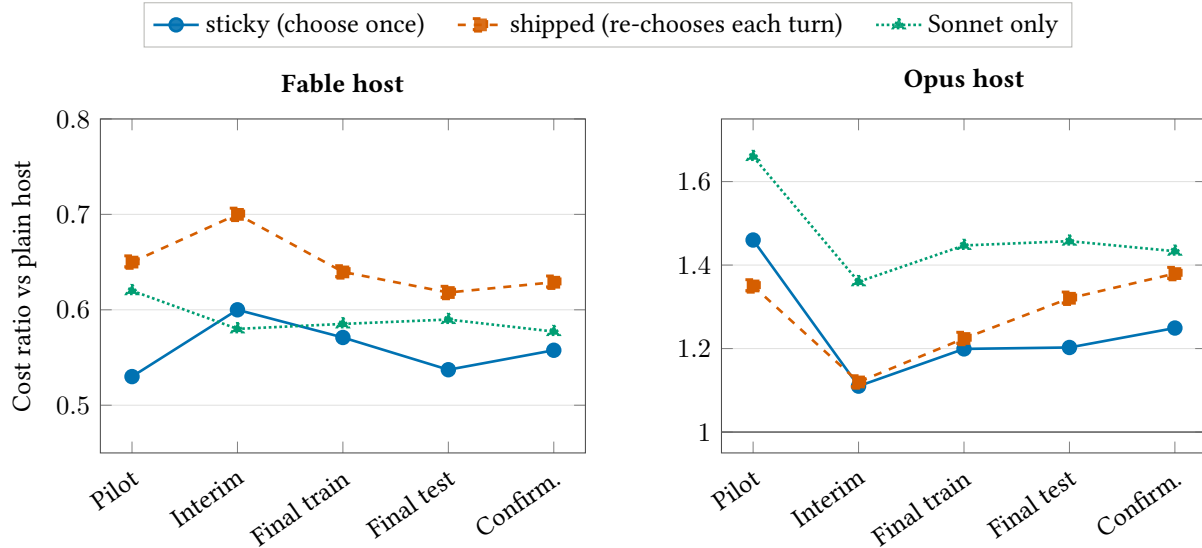


Figure 17: Cost ratio of each routed arm as the evidence accumulated: the screening pilot, the interim dashboard during the run (partial training data), the final training and test splits (all cost-valid pairs) and the confirmatory test result (pair reading).

How to read this figure. Read each line left to right. A flat line means the early numbers were already right. The Fable lines are nearly flat. The Opus lines jump: the pilot overstated the extra cost and the interim dashboard understated it.

On Fable, the pilot’s ratios (sticky 0.53, shipped 0.65) were already close to the final test values (0.54, 0.62). On Opus, the pilot overstated the penalty (sticky 1.46 against 1.25 in the confirmatory test) and the interim dashboard understated it (1.11). Small samples point in the right direction here but not at the right size, which is why the decision waited for the preregistered test.

10.6 Limits of the savings model

The model passed its preregistered check, and that check is about *pooled* accuracy: are the intervals right across all test pairs, and is the overall total inside its interval? It is not a check of each host and arm separately,

and the model’s structure makes per-host figures unreliable. Each arm effect and each task-type effect is shared by both hosts, although the data show the hosts behave very differently.

- Across the 42 host, task-type and arm cells, the model’s point prediction lies outside the measured 90 % range in 28 (Table 26).
- It ranks the arms wrongly within a host. On the Fable test mix it predicts that Sonnet saves \$1,549 per 1,000 sessions, less than shipped (\$2,082); the measurement shows the opposite, \$2,064 against \$1,848.

Use the model for what it was validated for, pooled forecasts over a mix like this campaign’s, and use the measured tables for any per-host, per-task or per-arm figure.

11 More explorations of the campaign data [\[screen\]](#)

Everything in this section is exploratory: it uses all 70 scenarios, was not preregistered, and is meant to show *how* the costs arise. It uses the 11576 turn rows of cost-valid sessions.

11.1 How cost accumulates over a session

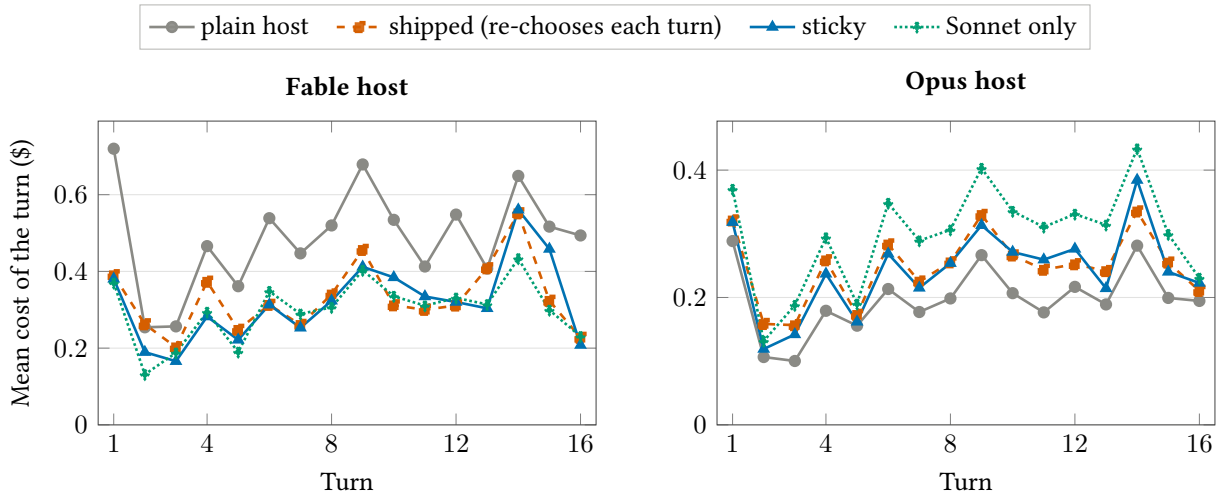


Figure 18: Mean cost of each turn by turn index, per arm and host. Later turns are averaged over fewer scenarios, because scenarios have 8 to 16 turns.

How to read this figure. Each line is one arm. Turn 1 is expensive for everyone (a cold cache); later turns are cheaper but grow as the history grows. On Fable the routed lines stay well below the plain host at every turn; on Opus they sit above it.

On the Fable host the first turn alone was 14 % of a plain session’s cost on average. [Figure 19](#) shows what each turn of a plain-host session paid for.

11.2 Long pauses and the cache

A pause longer than the cache lifetime should force the next request to write the whole history again. [Figure 20](#) confirms it: after a 7-minute pause, a plain Fable session’s first request wrote 65,062 tokens on average, against 11,296 after a 10-second pause ($5.8\times$; $6.2\times$ on Opus).

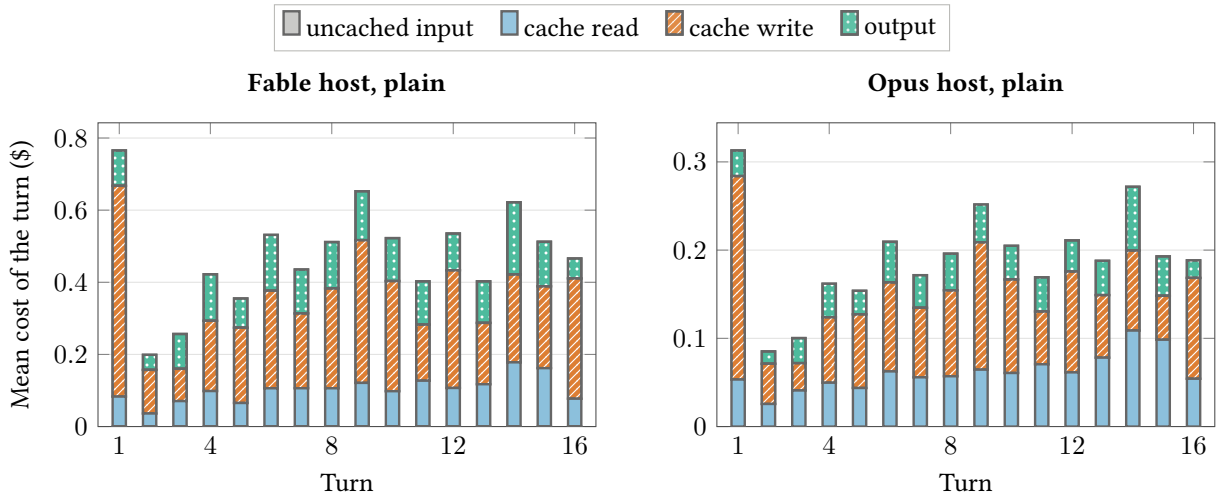


Figure 19: What each turn of a plain-host session paid for, by turn index (mean dollars per token class).

How to read this figure. Turn 1 is mostly cache writes (hatched): the whole prompt is new. Later turns are dominated by cache reads (blue) as the history is re-read each time, and the writes shrink to the new part of the conversation. Fable’s writes and output are expensive; Opus’s reads are cheap.

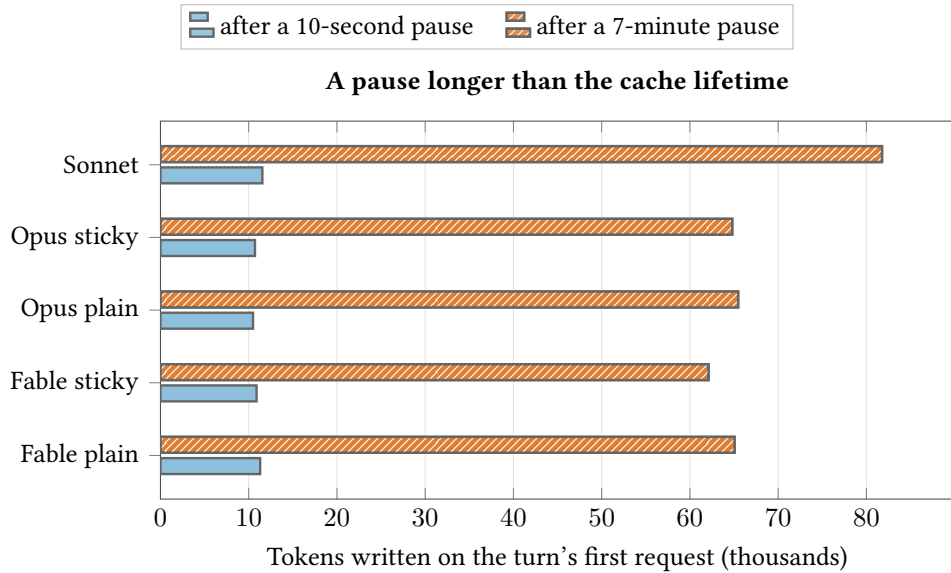


Figure 20: Cache-write tokens on the first request of a turn, after a short pause and after a long pause (turns 2 onwards), per arm and host.

How to read this figure. Each row has two bars. The hatched bar (after a long pause) is several times longer in every row: the expired history is written again, on any model.

11.3 When the shipped arm switched

The shipped arm re-chooses its model at the start of each turn. Its 422 switches were all at turn boundaries; 17 % of them led into turn 2 (Fig. 21). Each switch cost about 29.6 cents of rebuild on average.

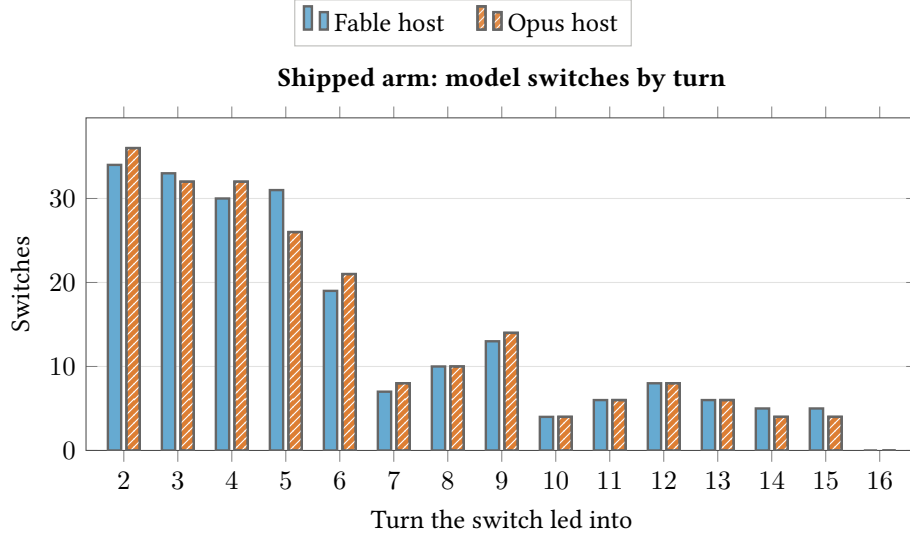


Figure 21: Shipped arm: number of model switches leading into each turn, by host.

How to read this figure. Each pair of bars is one turn. Switches are spread over the whole session rather than concentrated at the start; every one of them re-writes the history to the other model’s cache.

11.4 Scenario by scenario

11.5 The noise floor in detail

11.6 Quality by scenario family

Table 20: Mean difference in turn-pass fraction (arm minus plain host) by scenario family, all splits. Negative means the arm passed fewer scripted turns.

Family	scenarios	Fable host			Opus host		
		sticky	shipped	sonnet	sticky	shipped	sonnet
knowledge	14	−0.083	−0.061	−0.029	−0.060	−0.048	−0.018
mixed	20	0.009	0.001	0.015	0.010	−0.005	0.019
polyglot	20	−0.044	−0.025	−0.041	−0.007	−0.020	−0.000
repos	16	0.025	0.026	0.023	0.005	0.017	0.016

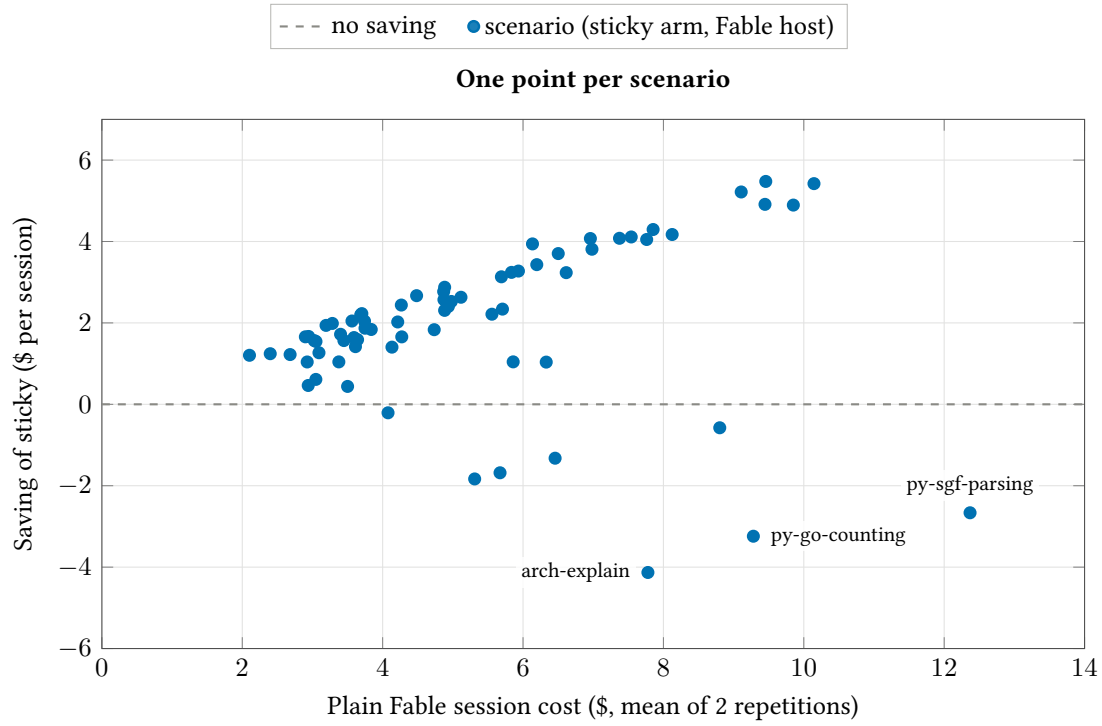


Figure 22: Scenario by scenario: the mean plain Fable cost against the mean saving of the sticky arm (Fable host). The three scenarios where sticky cost the most extra are labelled.

How to read this figure. Points above the dashed line saved money. Most scenarios did, and expensive scenarios tended to save more. 8 of 70 scenarios cost more on average; the labelled ones are the worst three.

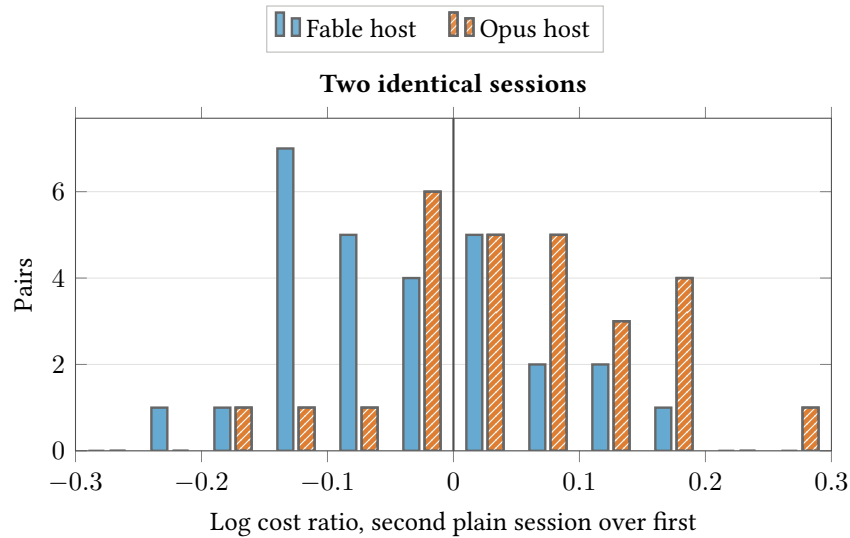


Figure 23: Distribution of the log cost ratio between two identical plain-host sessions (A/A), by host.

How to read this figure. A perfectly repeatable system would put every bar at zero. The spread is the noise floor; the Opus bars lean slightly to the right, the small bias discussed in [Section 10.4](#).

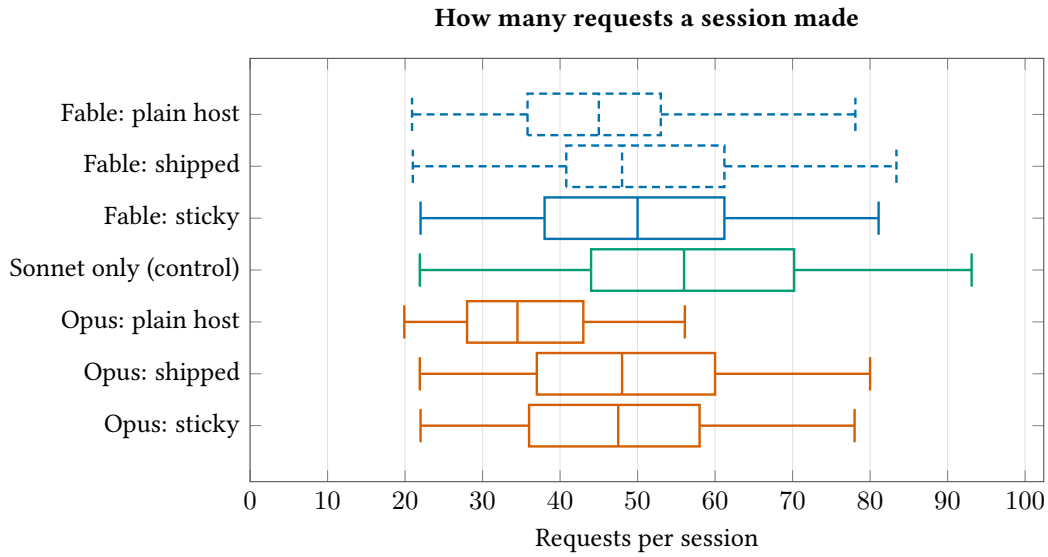


Figure 24: Main-loop requests per session by kind of session: box from the 25th to the 75th percentile, line at the median, whiskers from the 5th to the 95th percentile.

How to read this figure. Boxes further right made more requests for the same scripted work. Sonnet-heavy sessions sit to the right of the plain Opus sessions, which is part of why routing cost more on Opus.

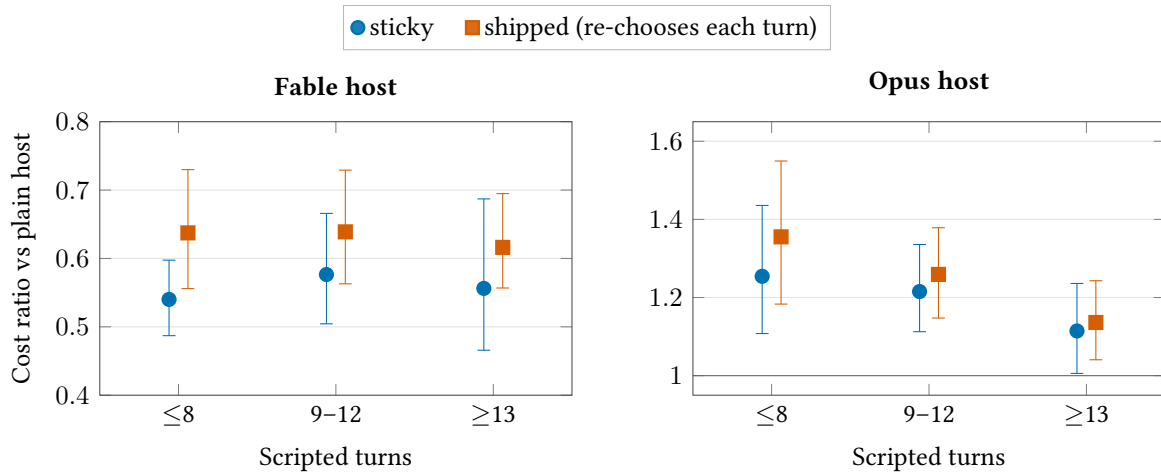


Figure 25: Cost ratio by scripted session length, all splits, with 95 % intervals.

How to read this figure. Flat rows mean the ratio does not depend on session length within 8–16 turns. Longer sessions moved the Opus ratios slightly toward 1; on Fable the saving held at every length.

11.7 Requests per session and session length

Part IV

Implications

12 What it means

Key idea

Table 5.1 at the Table 7 prices: move the session to Sonnet, and decide once. On Table 5.1 at the Table 7 prices, sticky routing saved about 44 %, with turn-pass fraction inside the preregistered 5-point margin on the 23 test scenarios. Across all 70 scenarios that margin was not shown for sticky (-0.021 , CI -0.058 to 0.012), and routed Fable arms passed the final hidden tests 3.6–4.3 points less often (exploratory). The sticky policy cost less than the shipped one ($0.862\times$ on Fable, $0.917\times$ on Opus); the preregistered test was only that sticky costs no more than shipped.

The data support “use Sonnet” as much as “use the router”. 248 of 280 sticky sessions ran on Sonnet only, so the sticky arm is mostly plain Sonnet. Plain Sonnet for the whole session cost about the same on Fable ($0.577\times$ against $0.558\times$ for sticky, test split) and saved more per 1,000 sessions over all scenarios (\$2,090 against \$1,980). What the router adds is the option to keep the host for the sessions it judges hard; this campaign does not show that option paying for itself. The judge’s own calls are not in the session costs (Section 7); by an estimate from another study they would add about 0.05 cents per shipped session, which does not change these conclusions. A fair reading of the evidence: on Table 5.1 at these prices, running the session on Sonnet saves about as much whether a judge decides or not, with the quality caveats above.

Key idea

Opus 5.5 at the Table 7 prices: do not route to Sonnet. Opus’s cache reads are cheaper than Sonnet’s, and Sonnet needed more requests for the same scripted work, so every routing arm cost more: 25–43 % more in the pair reading and 20–46 % in the arm reading. The plain host is the right default *at these prices*. If Opus cache reads cost about \$0.38 rather than \$0.20 per million tokens, sticky routing would roughly break even (Section 6).

Caching guidance (hypotheses). The mechanisms in Sections 6 and 10.1 suggest the following for other hosts and prices. They are hypotheses drawn from two hosts on one provider, not tested findings:

- **Compare price per token class, not price per token.** A model that is cheaper on input and output may be dearer in a long conversation if its cache reads cost more.
- **Every switch re-writes the history.** Switching less often, ideally once at the start, should avoid most rebuild cost.
- **Count requests, not just tokens.** The cheaper model made more requests for the same scripted work here; that volume, not only the price, decided the Opus result.

- **Pauses matter.** An idle gap longer than the cache lifetime makes the next request pay to write the history again, on any model.
- **Measure with isolation.** Give every measured session a private cache key (a nonce), and never start two identical requests at the same instant.

How to use the numbers. For a deployment that looks like this campaign, use the measured savings per 1,000 sessions in [Tables 18](#) and [26](#). The savings model is suitable for a pooled forecast over a similar mix, with its interval; do not use its per-host or per-task figures ([Section 10.6](#)).

What to measure next.

- **Longer sessions.** Real sessions often run for many dozens of turns; these ran at most 16.
- **Real traffic.** Scripted users do not wait, wander or abandon sessions as people do.
- **A production measure mode.** Routed and unrouted paths cannot both run in one live session, but each production session can carry the model's prediction and interval, and sampled offline replays can provide fresh paired measurements.
- **Other hosts and providers.** The direction of the result depends on the price table; a new host model needs its own measurement, or at least a check of its read and write prices against Sonnet's.
- **The final-pass signal.** A larger sample would settle whether the small excess of final-test losses on routed Fable arms is real ([Section 10.3](#)).

13 Limitations and threats to validity

- **Scripted, synthetic sessions.** The 70 scenarios are fixed scripts of 8–16 turns with automatic checks. Real users react to answers, pause irregularly and hold much longer conversations.
- **The judge's own cost is not measured.** Session costs exclude the routing judge's calls ([Section 7](#)).
- **Two hosts, one provider.** The conclusions are tied to these models and this price table; they do not transfer to other providers or price changes without new measurement.
- **A price table, not invoices.** Costs are tokens times list prices from the bundle's own table (verified 2026-06-10). They match the provider's per-request cost field exactly, but that field uses the same table, so the match checks token counting, not prices; discounts and batch pricing are not modelled.
- **The Opus result depends on one price.** At the table's \$0.20 per million cached Opus tokens, routing cost more; re-pricing the recorded tokens, sticky reaches parity at about \$0.38 (pair reading) or \$0.34 (arm reading), and at \$0.40 sticky would cost $0.98\times$ the plain host ([Table 8](#)). The Opus conclusion is about Opus 5.5 at these prices, not about "Opus-class" models in general.
- **The primary filter selects on an outcome.** The pair reading keeps pairs whose arm did at least as well as its anchor. That conditions on something the treatment affects. The arm reading, which keeps every cost-valid pair, gives the same verdicts; the filter raises the sticky and shipped ratios by 0.01 to 0.06 ([Table 14](#)).

- **Uncommitted harness code during the run.** The runner that executed the campaign was working-tree code, committed only afterwards (dc1aa4e). The agent under test was frozen at the preregistration commit, and re-extraction with the committed harness reproduces the rows byte for byte, but the exact runner code was not under version control while it ran.
- **The estimator was written after the data.** The preregistration fixed hypotheses, thresholds, split and decision rule; the confirmatory script that fixed resamples, seed, the quality filter and the verdict rule was committed about 56 minutes after the campaign finished, when exploratory summaries over all scenarios existed. The analysis also resolved 10 ambiguities or mismatches in the preregistration (Section B); only the two non-inferiority readings and the bootstrap seed were varied in sensitivity checks.
- **The Opus A/A ratio is not centred on 1, and launch order was fixed.** Two identical Opus sessions differed by about 5 % on average (1.049, 1.01–1.09). Arms were launched in a fixed order a few seconds apart (Section 7). Removing a bias of that size would put the Opus ratios at about 1.19–1.37; the verdicts hold.
- **Inferred failure causes.** For 8 of the 49 failed attempts the cause was inferred from timing, not recorded. Failed attempts were rerun as whole waves, so they cannot bias the accepted data, but the failure accounting for them is probable, not proven.
- **One shared API key.** Isolation between arms relies on the per-session nonce and the audit (0 flagged sessions), not on separate keys.
- **Quality is near the margin.** Fable sticky passed the turn-pass margin on the test split by 0.0035; over all scenarios the margin was not shown, and routed Fable arms passed the final hidden tests 3.6–4.3 points less often (exploratory, Section 10.3).
- **The savings model is additive.** It assumes the same arm and task effects on both hosts and misfits per-host cells (Section 10.6).

14 Reproducibility

Evidence. Everything is in docs/evidence/2026-10-02-paired-campaign/:

- data/ and its DATA-DICTIONARY.md, which documents every field: 1,036 session rows, 11,588 turn rows, 52,437 request rows and 896 pair rows;
- confirm/: the preregistered confirmatory analysis (confirm.json, CONFIRM.md);
- model/: the savings model, its summaries and its test-split predictions;
- campaign/: schedule, state, ledger, sticky decisions and FAILURES.md;
- prereg/: the preregistration, the split and PROVENANCE.md;
- pilot/ and reproduce/ (commands, versions, price table, spend).

The agent under test was frozen at commit 3aa2d1e (the preregistration commit) in every session.

Re-running the analysis. No model call is needed. From the published rows, the savings model re-fits byte for byte:

```
E=docs/evidence/2026-10-02-paired-campaign
S=$(mktemp -d); mkdir "$S/rows"
ln -s "$PWD/$E/data/sessions.jsonl" "$PWD/$E/data/pairs.jsonl" "$S/rows/"
nice -n 10 python3 evals/paired_model.py all "$S" --engine fallback
nice -n 10 python3 evals/paired_confirm.py "$S"
```

Re-extracting the rows themselves needs the raw campaign tree and per-session event files, which are not published; on the machine that ran the campaign, `evals/paired.py` rows reproduces them byte for byte.

This document. The source is in `docs/papers/2026-10-02-paired-measurement/`. Running `make` there runs `build_assets.py`, which reads only the committed files listed at its top (including the earlier caching survey from the `eval/judge-realistic` branch) and writes every number (`generated/numbers.tex`), table, plotted datum and label position, then builds the PDF. No number in the text is typed by hand. The script recovers the price table from the request rows themselves and checks that it prices every request to within 10^{-6} dollars. `make` check re-checks the figures' geometry on the built PDF: every scatter label nearest its own marker, every legend above its plot, and no overlapping text anywhere.

A Glossary

A/A

Running the same configuration twice on the same task to measure how much two identical sessions differ by chance.

Anchor

The plain host model with no routing; every other arm is compared with it.

Arm

One configuration of the agent in the experiment: `anchor`, `aa`, `shipped`, `sticky` or `sonnet`.

Bootstrap (scenario-cluster)

Estimating uncertainty by resampling whole scenarios with replacement many times and recomputing the result each time.

Cache read, cache write

Re-using a prefix already in the provider's cache is billed as a cheap read; storing a new prefix is billed as a more expensive write.

Cache rebuild

Re-writing the conversation history to a new model's cache after a switch.

Confidence interval (95 %)

The range of values consistent with the data; built so that over many repeats of the study it contains the true value 95 % of the time.

Confirmatory

A result that tests a hypothesis fixed in advance, on data held out for that purpose.

Exploratory

Any other result: useful for understanding, not for confirming.

Geometric mean

The average of ratios that treats “twice” and “half” symmetrically: the exponential of the mean of the log ratios.

Host model

The large model an agent session normally runs on (here Fable 5.1 or Opus 5.5).

McNemar’s test

A test for paired yes/no outcomes that looks only at the pairs where the two sides disagree.

Non-inferior

Not worse than the baseline by more than a set margin (here 5 points of turn-pass fraction).

Nonce

A unique string at the start of each session’s system prompt that makes its cache entries private.

Pair

A routed session and the anchor session on the same scenario, repetition and host.

Preregistration

A dated, committed plan of hypotheses, analysis and decision rules made before the data exist.

Prediction interval (90 %)

The range a new observation is expected to fall in 90 % of the time.

Routing

Sending some of a session’s requests to a different, usually cheaper, model.

Shipped

The bundle’s default routing policy; it re-chooses its model at the start of each turn and never switches in the middle of a turn.

Sticky

A policy that chooses the model once, from the first prompt, and never switches.

Tools-normalized cost

Cost with the shared tool definitions on a session’s first request priced as a cache read, as they would usually be in production.

Turn-pass fraction

The share of a session’s turns whose automatic checks passed.

Wave

All arms of one scenario, repetition and host, started together from the same workspace.

B Additional tables

Raw cost basis

Without the tools normalization the confirmatory ratios barely move: Fable sticky 0.552 (against 0.558) and Opus sticky 1.234 (against 1.249).

Table 21: Savings-model check by arm and host, test split: prediction-interval coverage and the observed total saving against its predicted 90 % interval.

Arm / host	pairs	coverage (log)	coverage (\$)	observed total	90 % PI of total	inside
shipped / Fable	46	0.957	0.978	\$85.01	64.78 to 121.24	✓
shipped / Opus	46	1.000	0.978	\$−31.48	−45.42 to −20.50	✓
sonnet / Fable	46	0.935	0.935	\$94.95	39.53 to 95.29	✓
sonnet / Opus	46	0.957	0.826	\$−46.80	−57.41 to −29.04	✓
sticky / Fable	46	0.913	0.935	\$99.82	70.67 to 132.14	✓
sticky / Opus	46	1.000	0.957	\$−21.15	−42.03 to −16.57	✓

Table 22: A/A noise floor: second plain-host session against the first. “SD” is the standard deviation of the log ratio per pair and per single session.

Split	Host	pairs	ratio	95 % CI	SD (pair)	SD (session)	centred
all	Fable	28	0.968	0.94–1.00	0.097	0.069	yes
all	Opus	27	1.049	1.01–1.09	0.103	0.073	no
all	both	55	1.007	0.97–1.04	0.107	0.076	yes
test	Fable	9	0.957	0.91–1.01	0.082	0.058	yes
test	Opus	9	1.035	0.96–1.10	0.103	0.073	yes
test	both	18	0.995	0.94–1.05	0.099	0.070	yes

Table 23: Interim and final cost ratios behind Fig. 17.

Host	Arm	pilot	interim	final train (95 % CI)	final test (95 % CI)	confirmatory
Fable 5.1	sticky	0.53	0.60	0.571 (0.51–0.64)	0.537 (0.48–0.61)	0.558
Fable 5.1	shipped	0.65	0.70	0.640 (0.58–0.70)	0.618 (0.55–0.70)	0.629
Fable 5.1	sonnet	0.62	0.58	0.585 (0.54–0.63)	0.590 (0.53–0.66)	0.577
Opus 5.5	sticky	1.46	1.11	1.199 (1.11–1.30)	1.203 (1.09–1.33)	1.249
Opus 5.5	shipped	1.35	1.12	1.224 (1.13–1.33)	1.321 (1.19–1.47)	1.380
Opus 5.5	sonnet	1.66	1.36	1.447 (1.31–1.60)	1.457 (1.29–1.67)	1.433

Table 24: Cost ratio by pause pattern, all splits (95 % CI). Exploratory.

Host	Arm	no long gap	two long gaps
Fable 5.1	sticky	0.56 (0.51–0.63)	0.55 (0.48–0.64)
	shipped	0.63 (0.57–0.69)	0.64 (0.57–0.73)
	sonnet	0.59 (0.55–0.65)	0.57 (0.53–0.62)
Opus 5.5	sticky	1.21 (1.11–1.32)	1.18 (1.07–1.30)
	shipped	1.30 (1.19–1.42)	1.17 (1.08–1.28)
	sonnet	1.47 (1.33–1.63)	1.41 (1.27–1.56)

Table 25: Cost ratio by scripted session length, all splits (95 % CI). Exploratory.

Host	Arm	≤8 turns	9–12 turns	≥13 turns
Fable 5.1	sticky	0.54 (0.49–0.60)	0.58 (0.50–0.67)	0.56 (0.47–0.69)
	shipped	0.64 (0.56–0.73)	0.64 (0.56–0.73)	0.62 (0.56–0.69)
	sonnet	0.63 (0.54–0.73)	0.57 (0.52–0.63)	0.57 (0.53–0.61)
Opus 5.5	sticky	1.25 (1.11–1.44)	1.22 (1.11–1.34)	1.11 (1.01–1.24)
	shipped	1.36 (1.18–1.55)	1.26 (1.15–1.38)	1.14 (1.04–1.24)
	sonnet	1.57 (1.34–1.85)	1.44 (1.29–1.61)	1.33 (1.18–1.50)

Deviations from the preregistration

Verbatim from `confirm/confirm.json`:

1. Coverage bounds: `paired_model.py` (PREREG constant) and `MODEL.md` use 0.83-0.97 plus a calibration-slope check 0.7-1.3; the preregistration says [0.80, 0.97] and has no slope check. The verdict here uses [0.80, 0.97]; the slope and `paired_model`'s own checks are reported, not used.
2. Coverage outcome: the prereg does not say which outcome's PI coverage is checked; both the log-ratio and the delta-\$ model coverage must be in bounds here (stricter of the two readings).
3. Primary-endpoint restriction is ambiguous (pair-level vs arm-level non-inferiority). Pair-level matches the text ('among pairs where ...') and is primary; arm-level is reported alongside. Pair-level filtering conditions on a post-treatment outcome and in this design keeps only pairs with `delta_turn_pass >= 0` (steps are $\geq 1/16$).
4. Non-inferiority comparison: `paired_model` uses lower bound ≥ -0.05 ; the prereg says > -0.05 , used here.
5. 'Lower bound' for Quality is the 2.5th percentile (two-sided 95% bootstrap CI), consistent with the 95% CIs of the primary endpoint; the prereg does not state the level for this bound.
6. 'Contradicted' is not defined in the prereg; defined here, before looking at test numbers, as the 95% CI lying entirely on the other side of the same preregistered threshold.
7. H3 restriction: the prereg does not say which pairs enter the sticky/shipped ratio; pair reading keeps triplets where both sticky and shipped are pair-level non-inferior to the shared anchor; arm reading keeps all.
8. Decision rule: 'H3 holds' is read as H3 as a whole (both hosts), and 'H1 holds' as both sticky and shipped.
9. Model engine: `statsmodels` is not installed, so the fit used `paired_model`'s numpy fallback (method-of-moments variance components + feasible GLS), not REML MixedLM. The model is the existing `model.json` (train only, fit-boot 400, seed 0); the PI check re-runs predict with its on-disk seed/draws and with seed 20261002 as a sensitivity check.
10. Bootstrap: 10,000 resamples, seed 20261002 (per analysis instructions; the prereg names 20261002 as the split seed).

Three further points, not listed in `confirm.json`:

1. **The estimator was written after the data.** `evals/paired_confirm.py` (commit c3ea41a) was committed about 56 minutes after the campaign finished, when the exploratory all-scenario summaries in `model/` already existed. It fixed the bootstrap (10,000 resamples, seed 20261002), the pair-level filter and the verdict rule, including "contradicted".
2. **What counts as a savings claim.** Only arm and host cells whose own cost interval shows a saving (upper end below 1) are treated as savings claims and tested for quality; on this data those are the three Fable cells.
3. **The design document's stricter model check also passes.** The design document asked for coverage between 0.83 and 0.97 and a calibration slope between 0.7 and 1.3; the model reached 0.960 (log ratio), 0.935 (dollars) and slope 0.83.

Table 26: Savings per 1,000 sessions by host, task type and arm, measured (all splits) and as predicted by the preregistered model. Positive numbers are money saved, negative numbers money lost. Exploratory.

Host	Task type	Arm	measured / 1,000	90 % range	model / 1,000	model inside
Fable 5.1	bugfix	sticky	1,346	848 to 1,882	222	✗
		shipped	797	385 to 1,232	64	✗
		sonnet	871	486 to 1,270	-352	✗
	feature	sticky	2,183	1,095 to 3,188	2,820	✓
		shipped	2,019	1,254 to 2,784	2,604	✓
		sonnet	2,895	2,350 to 3,484	2,031	✗
	mixed	sticky	2,020	1,241 to 2,720	2,771	✗

(continued)

Host	Task type	Arm	measured / 1,000	90 % range	model / 1,000	model inside
Opus 5.5	docs	shipped	1,941	1,532 to 2,355	2,553	✗
		sonnet	2,405	2,065 to 2,740	1,975	✗
		sticky	2,615	2,045 to 3,326	3,661	✗
	explain	shipped	2,263	1,924 to 2,683	3,445	✗
		sonnet	2,246	1,711 to 3,049	2,871	✓
		sticky	2,315	1,945 to 2,703	2,877	✗
	review	shipped	2,244	1,671 to 2,723	2,743	✗
		sonnet	2,367	2,058 to 2,668	2,388	✓
		sticky	3,007	2,469 to 3,509	3,853	✗
	all tasks	shipped	3,011	2,456 to 3,522	3,613	✗
		sonnet	2,389	1,863 to 2,797	2,977	✗
		sticky	1,980	1,577 to 2,368	2,531	✗
		shipped	1,754	1,481 to 2,032	2,314	✗
		sonnet	2,090	1,834 to 2,340	1,739	✗
Opus 5.5	bugfix	sticky	-1,037	-1,339 to -752	-1,112	✓
		shipped	-1,240	-1,501 to -996	-1,176	✓
		sonnet	-1,948	-2,300 to -1,626	-1,347	✗
	feature	sticky	-574	-776 to -385	-501	✓
		shipped	-490	-708 to -301	-590	✓
		sonnet	-1,099	-1,376 to -839	-824	✗
	mixed	sticky	-303	-511 to -104	-537	✗
		shipped	-401	-597 to -212	-626	✗
		sonnet	-815	-1,257 to -458	-862	✓
	docs	sticky	264	99 to 500	-159	✗
		shipped	162	1 to 321	-248	✗
		sonnet	-129	-458 to 185	-482	✗
	explain	sticky	341	227 to 434	150	✗
		shipped	235	14 to 397	95	✓
		sonnet	242	66 to 399	-50	✗
	review	sticky	-425	-838 to -29	-260	✓
		shipped	-642	-900 to -393	-358	✗
		sonnet	-1,193	-1,568 to -830	-618	✗
	all tasks	sticky	-489	-639 to -342	-625	✓
		shipped	-588	-726 to -456	-714	✓
		sonnet	-1,096	-1,329 to -881	-949	✓

Table 27: The 70 scenarios, their family, task type, language, number of scripted turns, number of long gaps and preregistered split.

Scenario	family	task type	language	turns	long gaps	split
commander-docs	knowledge	docs	javascript	13	2	train
debug-triage	knowledge	mixed	javascript	11	0	test
fastrand-explain	knowledge	explain	rust	9	0	test
fatihcolor-docs	knowledge	docs	go	8	0	train
fd-cli-docs	knowledge	docs	rust	16	2	train
filelock-explain	knowledge	explain	python	9	0	test
gohumanize-review	knowledge	review	go	12	0	train
humanize-review	knowledge	review	python	12	2	test

(continued)

Scenario	family	task type	language	turns	long gaps	split
kac-docs	knowledge	docs	ruby	14	0	train
ms-review	knowledge	review	javascript	8	0	train
nanoid-explain	knowledge	explain	javascript	10	0	train
pflag-mixed	knowledge	mixed	go	15	0	train
semver-review	knowledge	review	rust	10	0	test
yamllint-mixed	knowledge	mixed	python	14	2	train
api-docs	mixed	mixed	python	16	2	test
arch-explain	mixed	mixed	python	16	0	train
changelog-gen	mixed	mixed	python	12	0	train
config-matrix	mixed	mixed	python	12	0	train
deps-audit	mixed	mixed	python	8	0	train
discrepancy-docs	mixed	mixed	python	16	2	test
docstring-audit	mixed	mixed	python	12	0	test
expense-anomaly	mixed	mixed	python	12	2	train
hr-salary-qa	mixed	mixed	python	16	0	train
log-incident	mixed	mixed	python	8	2	train
meeting-notes	mixed	mixed	python	8	2	train
migration-plan	mixed	mixed	python	12	2	train
okr-status	mixed	mixed	python	12	0	train
pr-review-cart	mixed	mixed	python	12	2	train
pr-review-retry	mixed	mixed	python	8	0	train
readme-rewrite	mixed	mixed	python	8	0	test
sales-orders	mixed	mixed	python	16	0	test
support-workload	mixed	mixed	python	8	0	train
survey-analysis	mixed	mixed	python	16	0	test
timesheet-invoice	mixed	mixed	python	8	0	train
go-dominoes	polyglot	bugfix	go	13	2	train
go-linkedlist	polyglot	mixed	go	16	2	test
go-protein	polyglot	feature	go	8	0	test
go-say	polyglot	feature	go	11	0	train
go-vlq	polyglot	feature	go	12	0	train
js-listops	polyglot	feature	javascript	8	0	test
js-phone	polyglot	bugfix	javascript	8	0	test
js-rationals	polyglot	feature	javascript	12	2	train
js-tournament	polyglot	mixed	javascript	16	0	train
py-book-store	polyglot	bugfix	python	8	0	train
py-forth	polyglot	bugfix	python	10	2	train
py-go-counting	polyglot	mixed	python	14	2	test
py-poker	polyglot	feature	python	12	0	train
py-rest-api	polyglot	feature	python	9	0	test
py-sgf-parsing	polyglot	mixed	python	15	0	train
rust-gradeschool	polyglot	feature	rust	12	2	train
rust-luhn	polyglot	feature	rust	8	0	train
rust-ocr	polyglot	mixed	rust	16	2	test
rust-twobucket	polyglot	feature	rust	12	0	train
rust-wordy	polyglot	feature	rust	16	0	train
boltons-tableutils	repos	bugfix	python	8	0	train
cachetools-params	repos	feature	python	10	2	train
jmespath-strcmp	repos	bugfix	python	9	0	train
lark-discard	repos	bugfix	python	12	0	test
lark-template	repos	bugfix	python	8	2	test

(continued)

Scenario	family	task type	language	turns	long gaps	split
markdown-tocclass	repos	feature	python	10	2	train
mistune-escape	repos	bugfix	python	8	0	test
moreit-chunked	repos	bugfix	python	8	0	train
pathspec-matchfile	repos	bugfix	python	8	0	train
schema-wrongkey	repos	bugfix	python	12	0	train
sortedc-update	repos	bugfix	python	12	0	train
sqlparse-realname	repos	bugfix	python	8	0	train
tabulate-github	repos	bugfix	python	12	0	train
toolz-interpose	repos	bugfix	python	8	2	test
voluptuous-number	repos	bugfix	python	12	2	train
xmldict-none	repos	bugfix	python	8	0	test

Table 28: All 20 first-pass claims, what the first pass said, and what the validation found, followed by 1 entry recording changes merged after this study (verbatim from `changes.json`, where each entry also names its evidence file).

First-pass claim	First pass said	What the validation found	Verdict
GPT-6 Luna was right on all 90 cases	90/90	89/90 in the first pass itself when scored on its probabilities; 87-89 in 3 reproduction runs; 85-86 (majority 86/90) in the validated 3-rep run – the same as Jev (86/90, McNemar $p = 1.0$)	refuted
Luna made no wrong automatic decisions at any cutoff	0 at every cutoff	2 per repetition under the bundle policy (0-2 in reproduction); it clicked Buy now at $p = 0.96-0.99$ in all 3 validated repetitions after deferring in the first 4 runs	refuted
Jev is the best default: 95.6% correct, 147 ms, about \$16 per million	86/90, 4 wrong automatic at 0.75	85-86 per repetition in 7 runs (majority 86/90, 95% CI 89-98%); 2-3 wrong automatic per repetition under the bundle’s 0.90 gate; p50 152-163 ms; \$15.91 per million	confirmed
A stronger OpenAI model is not a better judge (Sol 88/90, both misses side effects at 0.98-0.99)	88/90	88/90 in all 7 runs; its misses are always cua-18 and cua-19; 13-23% of calls exceed the 3 s timeout, so bundle coverage is 52-66 of 90	confirmed
Best local judges reach 82-86%, not Jev’s 96%	Qwen3 8B 77, Qwen3 4B 76, nimble 74	identical answers in all 7 runs (fully deterministic); every local judge is less accurate than Jev after Holm correction (Holm-adjusted $p \leq 0.03$)	confirmed
tev1 0.8B is a credible fast tier with zero wrong automatic decisions from a 0.85 cutoff	0 wrong automatic at 0.85, 20 automatic	true only under the study rule, where a confident ‘no’ can abstain. The bundle acts on every yes/no answer, so under its policy tev1 0.8B makes 12 wrong automatic decisions of 90 (10 are accepted wrong code). With the no-abstention gap closed (I3) it automates under 30% of cases	refuted
Automatic means $p \geq 0.75$	study rule (stated choice, 0.75, confident no abstains, no timeout)	the bundle’s read-shortcut gate is $\text{argmax}, p \geq 0.90$ and $\text{margin} \geq 0.20$, 3 s timeout, and no abstention for yes/no; results are now reported under that policy, with the study rule kept for replay	revised

(continued)

First-pass claim	First pass said	What the validation found	Verdict
Local latency: tev1 0.8B 38 ms, nimble 157 ms, Laya 21 ms	single run	with identical answers, reproduction medians moved from 33% faster (Qwen3 8B) to 95% slower (Laya) on a loaded host (load average 15-36 when sampled); nonce-controlled warm medians: tev1 0.8B 48 ms, nimble 261 ms, Laya 46 ms via their decision endpoints	revised
Luna is about 8x slower than Jev because of OpenAI, not the client	1.19 s median; about 980 ms is server time	server time is 95% of the Luna-Jev gap and 98% of the Sol-Jev gap; the client adds about 1 ms; round trip to both hosts is about 13 ms	confirmed
service_tier priority cuts Luna's median about 14%	about 14% at 2x price	32% lower wall median (768 vs 1135 ms, 20 requests, one time window, not interleaved)	revised
Every judge except Luna fails on side-effect actions	Luna the exception	Luna also acts on side effects in the validated run; the root cause is the instruction, which lists writing and sending but not purchase or delete. A one-sentence clause removed all of Jev's side-effect errors on dev (5 to 0 over 3 repetitions); a deterministic host guard removed them for every judge at 1-4 points of coverage	revised
Labels are hand-authored, not independently audited	not audited	two blind reviewers agreed with all 90 frozen labels (Cohen's kappa 1.00). cua-18 and cua-19 depend on a rule the instructions do not state; fresh-cua-09 and search-11 are real failures	new
Order flips: Qwen3 0	0 flips	not comparable: the bundle's OllamaBackend averages both option orders inside every decision	revised
p95 latency	index $\text{int}(\text{round}(0.95 \cdot n)) - 1$	nearest rank; the first pass was one rank low for the 30-case slices (banker's rounding), unchanged for $n = 90$	revised
Laya base reproduces the earlier study (35/60)	35/60	35/60 on the original screen in all 7 runs	confirmed
The OpenAI Decisions API could not be measured	no endpoint visible	POST /v1/decisions returns 403 'Decision API is not enabled for this user' (logged in dev/run.json and holdout/run.json); the openai-decisions arm auto-enables on HTTP 200	confirmed
Cost per million decisions	Jev \$15.91, Luna \$34.95, Sol \$753.04	recomputed from logged tokens to the cent; Sol \$750.88 in the validated run	confirmed
Suggested routing: tev1 0.8B at a cutoff of 0.85 or higher as the offline fallback	zero wrong automatic from 0.85 (dev)	holdout: 35/63 correct and 14 wrong automatic decisions under the bundle policy; no local judge meets the preregistered offline-tier rule (best, tev1 4B: 3 wrong automatic at 44% coverage with guards, upper 95% bound 13% > 10%)	refuted
Keep Jev as the default judge	recommended	holdout (preregistered): Jev 55/63 correct, 1 wrong automatic; GPT-6 Luna 60/63 and GPT-6.1 Sol 62/63 are more accurate, but neither difference survives Holm correction ($p = 0.16$ and 0.09) and both fail the $p_{95} \leq 500$ ms and $\leq 2x$ cost rules. Jev's misses mostly fall back at low certainty; Luna's are confident (refund at 0.98-0.99, an injected submit at 0.97-0.99)	confirmed

(continued)

First-pass claim	First pass said	What the validation found	Verdict
Deterministic host guards on side-effect actions whatever judge is used	recommended, untested	preregistered intervention I2 confirmed on holdout: it removed every side-effect wrong automatic decision in all 9 applicable judges, with no accuracy change and 1.6-7.9 points of coverage	confirmed
<i>After this study (see Section 3.1); “first pass said” here means “before the change”</i>			
Changes since this study (main at 180f919, PR #56)	jev_cua gate 0.75 with no margin; no side-effect guard or clause in jev_cua; the local backend dropped a question’s own reason option (worked around in the harness)	PR #56, merged after these measurements: jev_cua uses the read-shortcut gate (0.90 with a 0.20 margin, configurable), refuses targets whose label names an irreversible or external change (the I2 guard, returning side_effect_requires_confirmation), names those changes in its instructions (I1), and the local backend answers a question’s own reason option again. jevgrep and jev_cua descriptions now say Jev is the default. Results in this directory are unchanged: they were measured before the merge.	new